WebTV™ Service Software Overview

Part Number 980-01001-A10

Copyright 1997 WebTV Networks, Incorporated. All rights reserved.

This material constitutes confidential and proprietary information of WebTV Networks Incorporated. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior written consent of the publisher.

WebTV and WebTV Network are trademarks of WebTV Networks, Inc.

This publication and the information herein is furnished AS IS, is subject to change without notice, and should not be construed as a commitment by WebTV Networks Incorporated. WebTV Networks Incorporated assumes no responsibility or liability for any errors or inaccuracies, makes no warranty of any kind (express, implied or statutory) with respect to this publication, and expressly disclaims any and all warranties of merchantability, fitness for particular purposes and of third party right.

Contents

Preface

This Guide Contains ix
Conventions Used in This Guide x

1 System Overview

System Software Overview 2
Access Services 3
Proxy Services 3
Messaging Services 4
Content Services 4
Miscellaneous Services 5
Administration and Monitoring Services 6
Back-end Services 7
Network Configuration 7
Service Networks 8
The modem Net 8
The Nexus Net 8
The db Net 9
The IAP /ISP Net 9
The Admin Net and Console Service 9
The Store Net 9
Service Configuration 10
Service Operation 10
Registration 10
Normal Use 12

2 Service Files

Configuration File 15
File Organization 16
Global Settings 16
Service Settings 16
Test Settings 17
Syntax 17
HTML Template Files 18
Template Syntax 18
{ } 18
LABEL 19
INCLUDE 19
IF or ELSE 19
IFDEF 19
IFNDEF 20
DEFINE 20
FILE 20
An Example Template File 20
Service Log Files 22
Directory Structure 22
Major Log Files 22
Typical Log Problems 23
Log File Format 23
Parts of the Log File 23
An Example Log File 24

3 WTVP Protocol

Components of WTVP 26
WTVP Headers 26
Standard HTTP Headers 26
WTVP Attributes 26
WTVP Security 29
WTVP Server-signed Headers 29
Debugging Encryption-Caused Problems 29

4 Access Services

Headwaiter Service 31
Headwaiter Service Components 31
Headwaiter Operation 32
Special Files 34
Error Log 34
Events Log 34
Configuration files 34
Star Service 35
Star System Components 35
Star Operation 36
Special Files 36

Error Log 36
Events Log 36
Configuration file 37
Templates 37

5 Messaging Services

Mail System 39
Mail System Components 40
mailsorter server 40
postoffice Server 40
Mail Store Server 41
notify Server 41
mailtod Server 41
Customer Database 41
Mail Operation 41
Receiving Mail 41
Listing and Reading Mail 41
Moving or Deleting Mail 42
Sending, Forwarding, or Answering Mail 42
Managing an Address Book 42
Special Files 42
Log Files 42
Configuration files 42
Software Tools 43

6 Proxy Services

Proxy Service 45
Proxy Service Operation 45
FTP and Gopher Requests 46
HTTP Requests 46
Special Files 47
Log Files 48
Configuration Files 48
Cookie Service 49
Cookie System Components 49
Cookie Operation 50
Setting a cookie 50
Getting a cookie 50
Special Files 50
Error Log 50
Events Log 51
Configuration Files 51
Service Tests 51

7 Content Services

Guide Service 53
Guide System Components 53

Guide Operation....54
Special Files....54
Error Log 54
Events Log 54
Configuration files 55
Templates 55
Explore Service 55
Explore System Components 56
Explore Operation....56
Special Files....57
Log Files.... 57
Configuration File 57
Home Service 57
Home System Components....58
Home Operation....59
Special Files....59
Log Files.... 59
Configuration File 60
Content Service 60
Content System Components....60
Content Operation....61
Special Files....61
Error Log 61
Events Log 61
Configuration File 62
Aroundtown Service 62
Aroundtown System Components 63
Aroundtown Operation....63
Special Files....64
Error Log 64
Events Log 64
Configuration file 64
Service Tests 65

8 Miscellaneous Services

Favorites Service 68
Favorites System Components....68
Favorites Operation 69
Saving a favorite 70
Using a favorite 70
Special Files....70
Error Log 70
Events Log 70
Configuration File 70
Templates 71
Service Tests 71

Tricks Service 71
Special Files 72
Error Log 72
Events Log 72
Configuration File 73
Templates 73
Setup Service 73
Setup System Components 74
Setup Operation 75
Special Files 75
Error Log 75
Events Log 75
Configuration File 76
Templates 77
Service Tests 77
Flashrom Service 77
Flashrom System Components 78
Flashrom Operation 79
Upgrading the Flash ROM 79
Replacing a Corrupt Flash ROM 80
Special Files 80
Error Log 80
Events Log 80
Configuration Files 81
FlashROM Images 83
Templates 84
Software Tools 85
Service Tests 85

Glossary

Preface

This book provides an overview of the components that make up the WebTV system software. It provides an overview of the system as well as specific details on each component that makes up the system.

This Guide Contains

"Chapter 1, "System Overview," describes the parts that make up the WebTV system and how the system works in general.

Chapter 2, "Service Files," provides an overview of the configuration files, log files, and templates that each service uses.

Chapter 3, "WTVP Protocol," provides an overview of the structure, debugging tricks, and security layer of the WTV protocol.

Chapter 4, "Access Services," provides an overview services that handle preregistering users, registering users, validating and greeting users, notifying users of problems, and logging user's movements.

Chapter 5, "Messaging Services," provides an overview of services that handle all forms of messaging, including sending and receiving mail and reading and posting to news groups.

Chapter 6, "Proxy Services," provides an overview of services that handle HTTP, FTP, and gopher protocols as well as maintaining a user's preferences for specific Web sites they may visit.

Chapter 7, "Content Services," provides an overview of content services that a user has access to when using the Internet terminal.

Chapter 8, "Miscellaneous Services," provides an overview of miscellaneous services, which include the services that handle changing the configuration of the WebTV services and bookmarking favorite Web pages.

Conventions Used in This Guide

The following typographical conventions are used in this book:

Typeface	Description	Example
`AaBbCcDd`	The names of tags, attributes, directories, on-screen computer output	The WebTV interface implements the `<!-->` comment tag.
AaBbCcDd	Placeholder: replace with a real name or value	`<a href="`*attribute*`">`
AaBbCcDd	Book titles, new words or terms, or words to be emphasized	Using the *WebTV Internet Terminal*, consumers...

Chapter 1

System Overview

This chapter provides an overview of the WebTV™ network and the services that make up the system. It also describes the typical configuration of servers for each service. It contains the following sections:

- **System Software Overview**—contains a block diagram of the service and briefly describes each component
- **Network Configuration**—contains a description of the networks that provide the WebTV services
- **Service Operation**—describes the basic operation of the service; that is, how the system registers users and how the system operates for a normal user's session

System Software Overview

WebTV network services are divided into seven types: access services, proxy services, messaging services, content services, miscellaneous services, back-end services, and administration and monitoring services. Figure 1-1 shows the overall architecture of the software system. The parts of the system are described following the figure.

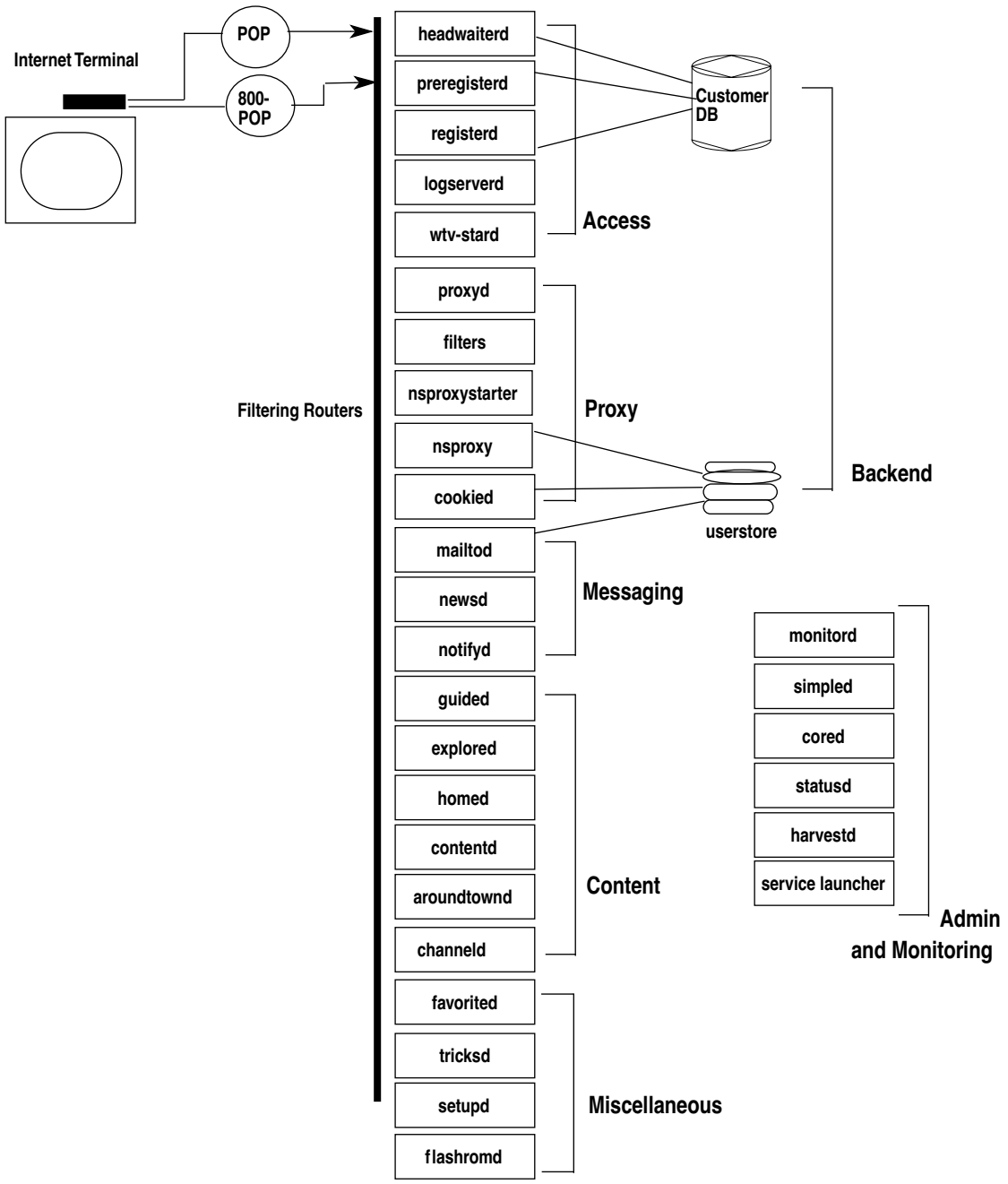


Figure 1-1 Overview of WebTV Services

Access Services

Access services include services that handle preregistering users, registering users, validating and greeting users, notifying users of problems, and logging user movements.

Headwaiter Service

Headwaiter (headwaiterd) greets terminals wishing to enter the WebTV service, authenticates or logs them into the service, and assigns them services.

Preregistration Service

Preregistration (preregistrationd) is used the first time the subscriber dials in to WebTV or if the subscriber moves the Internet terminal to a new location and selects "New Number." preregistrationd determines the telephone number and downloads information to the Internet terminal that it will later need to communicate with the WebTV service.

Registration Service

Registration (registrationd) registers the subscriber with the WebTV service by asking for a user name, billing information, and so on.

Logserver Service

Logserver(logserverd) keeps track of the connection logs that are uploaded when the terminal connects to the service. The terminal uploads a record of everything it has tried to dial since its last successful connection and stores this information in the logserverd.log.

wtv-star Service

When the WebTV Service is unavailable and users attempt to connect, they are directed to the Star (stard) service. Star offers users explanations and possible options when connection is impossible.

Proxy Services

Proxy services include services that handle HTTP, FTP, and gopher protocols as well as maintaining a user's preferences for specific Web sites they may visit.

Proxy Service

Proxy (proxyd) translates HTML, the net-standard HyperText Markup Language spoken by the majority of the Web, into formats better handled by WebTV devices. proxyd also translates HTTP into WTVP, the compatible WebTV Protocol and converts data into more compact forms.

Filters

WebTV uses Surfwatch filters to provide two levels of restriction: Blacklist and Whitelist. The Surfwatch account mode allows access to most of the Internet but restricts access to certain accounts. This is analogous to X-ratings. The most restrictive type of account is "kid-friendly," which only allows access to specific sites, excluding everything else. Kid-friendly accounts are intended for customers under 12-years old.

Netscape Proxy

WebTV uses Netscape proxy software to handle ftp and gopher requests. When the proxy software receives such a request, it sends it to nsproxy.

nsproxystarter

Software that starts the Netscape proxy.

Cookie Service

Cookie (cookied) accepts, stores, and retrieves “cookies” requested by the terminal. Cookies contain personalized information about the user, such as color preferences and passwords, that a remote web service intends users to store locally on their machine. The remote web service can then retrieve this information for its use when the user contacts that web service in the future. The Cookie service maintains separate cookie information for each user; the actual cookie data is stored on a user store.

Messaging Services

Messaging services include services that handle all forms of messaging, including sending and receiving mail and reading and posting to news groups.

Mail Service

Mail (mailtod) is the interface between sendmail, the UNIX® mail-handling application, and the terminal. mailtod provides users with their mail, stored on a mail store, and handles the transfer of outgoing email to sendmail.

News Service

News (newsd) is the interface between the NNTP server and the terminal. This service lets the user read and post net news.

Notify Service

Notify (notifyd) is used by other servers to asynchronously control terminals. Any service can deliver a message for a specific user to notifyd, and notifyd then delivers the message to the terminal. For example, when a user receives a mail message, the mail.local.webtv program uses notifyd to tell the terminal to activate its red message light.

Content Services

Content services include all of the services that a user has access to when using the Internet terminal.

Guide Service

Guide (guided) is a service whose only purpose is to serve sequenced pages of instructions to users. Home Page, Mail, Explorer, and Search all have instruction links in their sidebars that connect to the Guide service. However, the guide service does not interact with any other service.

Explore Service

Explore (explored) is a service that serves a small directory tree on demand. The tree reflects categorized sites and descriptions that WebTV reviewers have selected. The goal of Explore is not to catalog the Internet, but rather to provide a simple, straightforward categorization of content to users new to the Internet. As users navigate, history data is saved on a user-store, allowing Explore to behave in a more personal fashion.

Explore for Kids is a subset of Explore that does not allow access to other category topics. As the “kid-friendly” filter is expanded in the future, this restriction will go away.

In addition to categorizing Web sites, Explore will be used to categorize news groups and chat rooms in future releases.

Home Service

Home (homed) serves the main home page for WebTV customers. The home page is split into several content areas. The large, black-background section of the home page is considered the “dynamic” part and changes daily. Customers with kid-friendly accounts see a special version of the home page. For testing purposes, home (in service 1.3.8 or higher) allows previewing of particular days. For example, to see Wednesday's home page, the URL is `wtv-home:home?wednesday`.

The homed server determines what to present, based on subscriber information (terminal version, account type). For example, a 1.0 box with a single user has “Setup” and “Instructions” in the sidebar. A 1.1 box with multiple users has “Setup,” “Instructions,” and “Switch User” in the sidebar.

Content Service

The Content service (contentd) serves plain web content to users. It has no program functions apart from serving static content. Typical content might include information about the latest upgrade of the service, a guided tour of the service, and so on. The Content service is used like an ordinary web server and runs on the storeless machine.

AroundTown Service

AroundTown (aroundtownd) serves local content to customers. The AroundTown server dynamically determines some of the URLs for the icons featured on its page; for example, MovieLink, Weather, or Yahoo. The customer’s subscriber information (from the Customer DB)—City Name, State, Zip—is used to customize content. AroundTown can determine the customer’s closest metropolitan area name; for example, Dallas, based on the subscriber ‘s zip code. In future versions of AroundTown, the ANI-zip code will be used instead.

Channel Service

Channel (channeld) serves “Channel” content. Channel content is typically content that is aggregated in some fashion by WebTV and presented to the user in an organized manner. The first channel in development is the “Sports” channel. For testing purposes, channeld (in service 1.3.8 or higher) allows previewing of particular days. For example, to see Wednesday’s channel, the URL is `wtv-channel:channel?wednesday`.

Miscellaneous Services

Miscellaneous services include the services that handle bookmarking favorite Web pages and changing the configuration of the WebTV services.

Favorites Service

Favorites (favorited) tracks a user’s “Favorites,” or preferred URL pages recorded in the form of bookmarks. favorited both provides users with their Favorites on login and acts to record additions to the list of Favorites. Each user has an individual Favorites list; the actual Favorites data is stored on a user store.

Tricks Service

Tricks (tricksd) is a password-protected service to allow WebTV employees access to special functions, such as requesting a download of a particular version of the system software or starting the tourist (a test tool that visits many web sites).

Setup Service

Setup (setupd) lets the user customize the WebTV service. For example, it lets users choose the type of keyboard that they want to have displayed, specify the way they want to display mail messages, and so on.

Flashrom Service

Flashrom (flashromd) updates the Internet terminal with new software when there is a new release.

Administration and Monitoring Services

Admin and Monitoring services include all of the services necessary to monitor and administer services; for example checking to see that a service is running.

Monitord Service

Monitord periodically connects to each service and runs several tests. If the service does not pass the test, a message is sent to a warning address or a failure address which in turn is connected to pagers that operations persons carry.

simpled Service

simpled is a minimal web server, useful for serving web content. It is currently use it to display web pages containing usage statistics reports.

Cored Service

Cored is a program that renames core files generated when a service crashes so that the files aren't overwritten by subsequent core files. The core file names are in the format of `core.programName-YYMMDD-HHMMSS`. For example, `core.cored-970122-113453`.

Statusd Service

statusd runs on each production machine. It periodically is contacted by monitord to monitor various system-level resources, such as CPU load and disk fullness.

Harvest Service

The Harvest service (harvestd) retrieves log files from each machine, trims them down, and produces a variety of graphs that can be used to determine if there is a problem. The most useful information it provides pertains to relative loads for each POP, which isolates places where increased coverage is necessary.

Service Launcher

The service launcher starts all of the services that need to run on a given server. If any of these processes dies, the service launcher restarts it.

Back-end Services

Back-end services include the storage of database and user information.

Customer Database

The customer database stores administrative and billing data about users. It holds the user's phone number and Internet terminal's encryption key; both are obtained during preregistration. The customer database is currently stored in an Oracle® database and will eventually be stored on a pair of Oracle database servers, using the Symmetric Replication™ feature.

userstore

The user store is an area in the file system that contains user specific information in the form of files. Different applications typically have different user store areas, each containing user information for a specific service. For example, the users' favorites are stored in the favorites user store area. Usually the user store resides on a NFS® (Network File System) file server, often on a dedicated Network Appliance (NetApp) server.

Network Configuration

The WebTV service is made up of several networks; each network connects only those machines needed to provide a related set of services. Figure 1-2 shows a typical network configuration. The parts of the network are described after the figure.

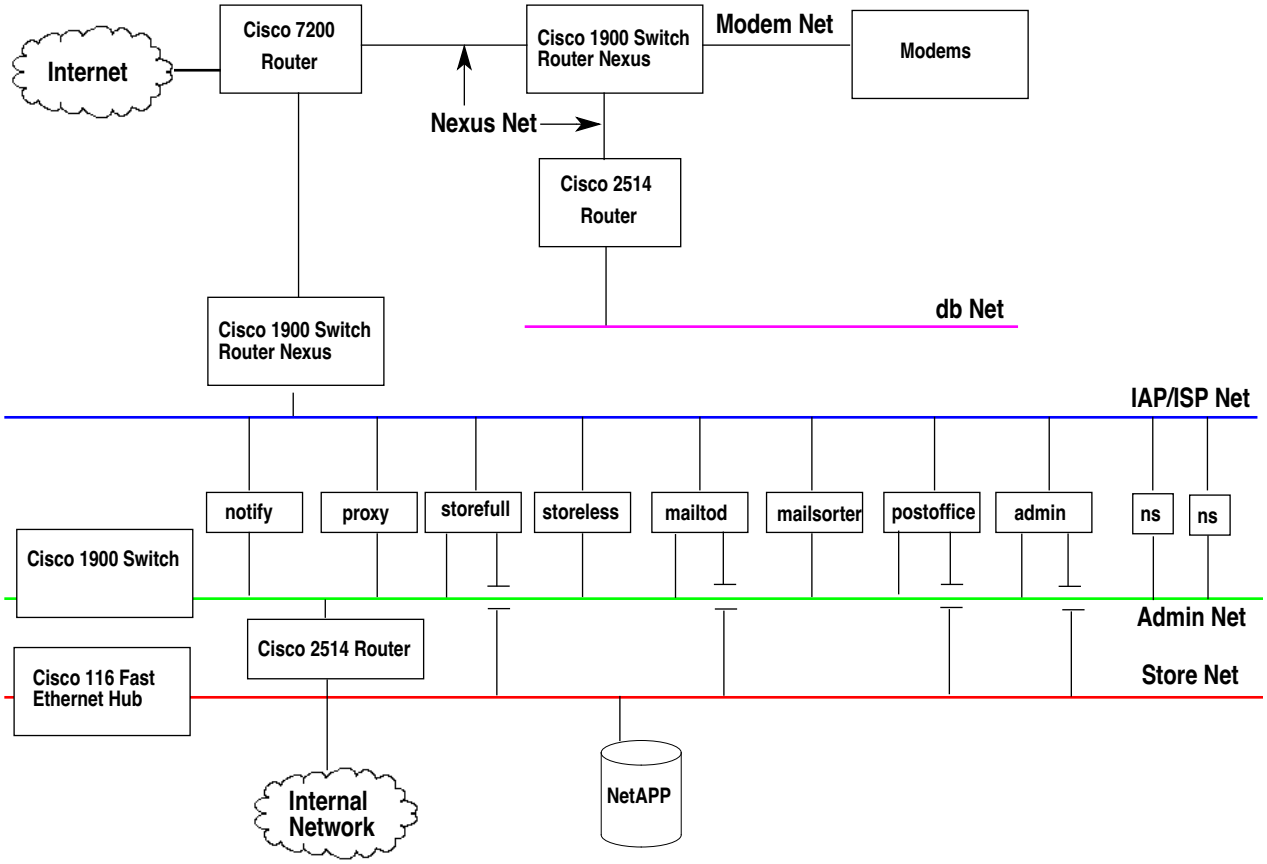


Figure 1-2 Network Configuration

Service Networks

A typical service network consists of a modem net, Nexus Net, db Net, IAP/ISP Net, Admin Net, and Store Net. Each network is described below.

The modem Net

US Robotics® modem banks hold the modems that provide both the preregistration service and the backup access for customers whose local Internet providers could not be contacted. The network that hosts the modem banks is connected to the Nexus Net.

The Nexus Net

Other networks are needed to run the WebTV service; a single network, called the Nexus, serves as the central connection point where these networks meet the IAP/ISP network. In general, only routers are connected to the Nexus Net; the Internet router(s) are connected here, but no other IAP/ISP services have interfaces on this net.

The db Net

Two hosts are assigned to run the database applications that support the WebTV service: custmaster and custservice. These machines are connected to each other by the db Net. This network is attached to the Nexus Net by two small firewall routers. The access lists of these routers provide an extra level of security for the database machines.

The IAP /ISP Net

The heart of the WebTV service is the Internet Access Provider/Internet Service Provider (IAP/ISP) network. This network forms the link between the Internet router(s) and every server that might have to communicate with the Internet terminal. All proxy, storefull, storeless, mailtod, mailsorter, postoffice, and notify machines are connected to this network.

If some of the other processes run on dedicated machines, those machines also have an interface to the IAP/ISP network. Examples include ns servers for DNS service, realaudio servers to run RealAudio, and prereg servers for preregistrationd. In other cases, these processes have been assigned to share the storefull, storeless, and other servers.

Finally, the monitoring and administration machines—the log processing and admin servers—are connected to IAP/ISP network.

The Admin Net and Console Service

Access for administration of the machines on the IAP/ISP Net and Store Nets is provided in two ways. Each machine has an interface on the Admin Net, which also hosts several systems dedicated to administration and monitoring of the service machines. The Admin net is also connected to a router that serves as a gateway to the internal network of the company.

In addition, each production machine and each of the routers, network switches, and hubs are connected through its console port to a remote terminal server allowing system administrators access to the machines.

The Store Net

Many of the WebTV service servers make no use of access to shared or permanent storage. Each proxy server, for example, has its own attached disk storage to hold cached data. Some servers, however, need to store and retrieve electronic mail, cookies, and favorites. Because this data needs to be accessible to all postoffice, mailtod, and storefull servers and because it needs to persist between customer sessions, it is stored on dedicated file servers.

The file servers are Network Appliance machines, which are optimized to provide extremely fast Network File System (NFS) service. The Store Net connects these two file servers with the production servers that need file service.

Keeping the store net separate from the IAP and ISP network has two major advantages: high-speed NFS access does not compete for bandwidth with the rest of the service transactions, and private user data is one hop farther away from possible attempts to compromise it.

Service Configuration

The WebTV services that implement WebTV run on UNIX servers that are configured and networked for optimal performance and security.

All servers run statusd and service launcher. Each server also includes a configuration file called arcadia.cf and a variety of logfiles appropriate to the functions that they serve; these files are described in more detail in Chapter 2. The configuration file maps each service to its role, and each role to the set of processes it must execute. The logfiles record events that pertain to their services.

Some services run on their own server and others are grouped onto one of three servers: Storeless, Storefull, and Admin. Services that run on Storeless servers do not need access to a user store; services that run on Storefull services do need access to a user store. Services that run on Admin servers monitor and administer other servers.

- Proxy servers include proxyd, and they interpret http, ftp, and gopher.
- Mailtod servers include mailtod and sendmail.
- Postoffice servers include mail.local.webtv and sendmail processes.
- The Mail Sorter server includes sendmail and aliasd.
- Storeless services include aroundtownd, channeld, contentd, flashromd, guided, harvestd, homed, searchd, and tricksd.
- Storefull services include cookied, explored, favorited, flashromd, harvestd, registerd, and setupd.
- Admin services include a monitord and anid and harvesterd.
- Notify server includes the notifyd service.
- WTV-star includes the stard service.

Service Operation

The service operation is composed of two types of activities: 1) registering with WebTV for the first time or reregistering when the terminal has been moved to another location and 2) normal use of the services.

Registration

The registration process includes preregistering the user (determining the subscribers' phone number and local *Point of Presence* (POP)) and then registering the subscriber (determining the user name, payment method, etc.). Figure 1-3 shows the registration process.

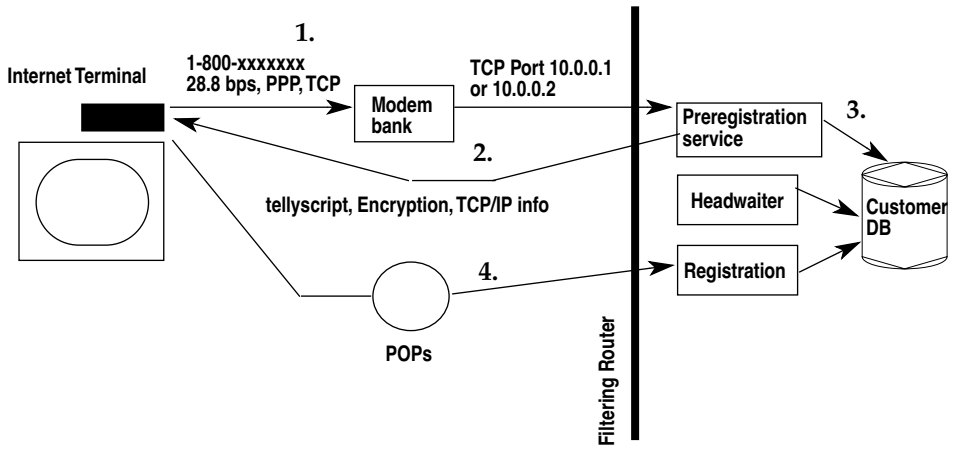


Figure 1-3 Registration Process

1) When the user buys a WebTV Internet terminal, takes it home, and dials out for the first time, the terminal has only a hardcoded 800 number and a pair of IP addresses embedded in it. The terminal uses this number to dial into a bank of modems; the modems establish a 28.8 kilobit per second PPP connection with the terminal, which then establishes a TCP/IP connection to a preregistration (preregistrationd) service running on a UNIX machine. The preregistrationd service verifies that the call is coming from a WebTV terminal and connects to an ANI (Automatic Number Identification) server. The anid determines where the terminal is calling from and returns that information to preregistrationd. preregistrationd uses that information 2) to download three pieces of information to the terminal:

- tellyscript—a small program that the terminal can run to dial into a POP. Each tellyscript contains the appropriate (cheapest) local phone numbers of one or more POPs as well as the fallback toll-free number.
- Encryption key—a key that lets passwords and encrypted data be deciphered. The encryption key is also used by the Headwaiter service when the terminal connects to the WebTV service.
- TCP/IP information—the IP address and TCP port numbers of other processes at WebTV with which the terminal will need to communicate.

3) When the download is complete, preregistrationd sends the user's phone number and the encryption key to the customer database for storage.

4) With preregistration complete, the terminal disconnects from WebTV's toll-free number and uses the tellyscript program to dial into an available POP. It tries two POPs, twice each. If these calls go unanswered or are busy, the terminal calls a toll-free phone number at WebTV and makes its connection.

Note: When the terminal loses and regains electrical power, the user is asked if the terminal is connected to a different phone number. If the answer is yes, the preregistration process is repeated. When WebTV wants to provide a new or different local phone number, a new tellyscript program is downloaded from the Headwaiter to the terminal.

When the terminal connects through the Internet to WebTV, it connects to a headwaiter server, using an IP address and TCP port number obtained through preregistration. All WebTV services come in pairs, so if for some reason the terminal's chosen headwaiterd process didn't respond, the terminal asks for its second choice. The Headwaiter authenticates the terminal by checking its serial number in the customer database and checking the encryption key. Because this is the first time the terminal has connected to WebTV, the Headwaiter refers it to the registration service (*registrationd*.). This service asks the subscriber to choose a user name and (optionally) a password. It also requests the subscriber's name, address, telephone number, and billing information, including a credit card number.

Normal Use

Once the user has registered, the next time 1) that user connects to WebTV, 2) headwaiterd authenticates the terminal, using the serial number and encryption code. headwaiterd then checks the status of the customer's account to see if it is necessary to update the billing information. If so, the user is directed to call the customer center. If the terminal needs to be updated, headwaiterd asks a process called flashromd to provide the new software. 3) Finally, the Headwaiter passes a list of available services, including each service's name, host address, port number, and other information to the terminal. It distributes the addresses in a round-robin fashion, providing a simple load-balancing facility. Figure 1-4 shows the flow of normal service.

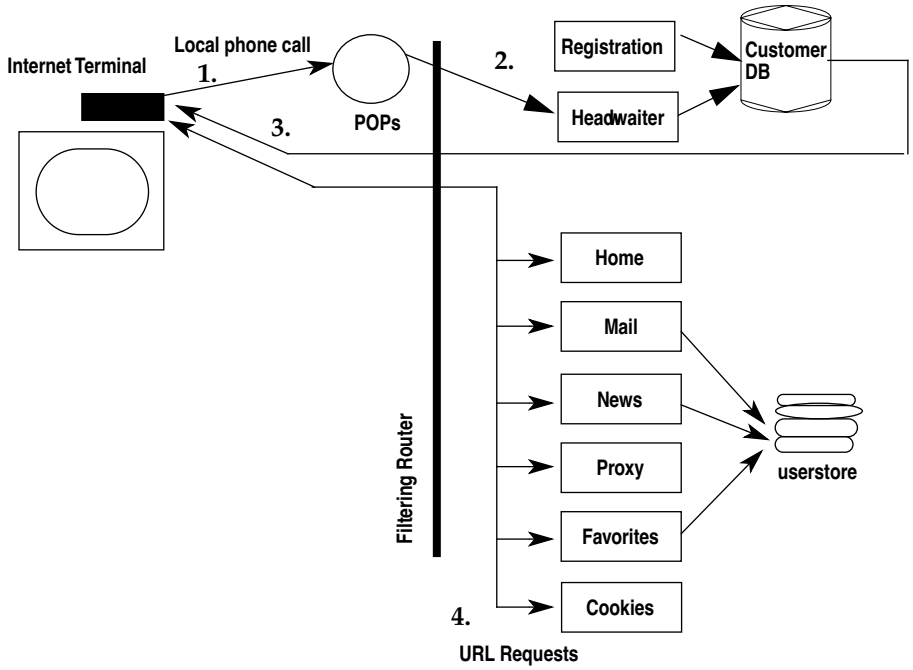


Figure 1-4 Normal Service

4) Users are then able to use any of the normal services such as email or navigating to their favorite URL. All URL requests are handled by the proxy (proxyd) service. The proxy service takes the URL requests from the user, issues

them through one of WebTV's Internet access providers, and displays the results for the user. proxyd also caches the page for faster response, compacts certain kinds of data, refuses to transfer adult data when the terminal is in Kidsafe mode, and translates certain pages that would otherwise report errors to the customer.

When the WebTV server needs to conserve bandwidth or the terminal can't reach WebTV, the terminal can be told to contact sites directly, rather than going through WebTV and proxyd. However, the performance advantage of caching and compacting provided by proxyd is lost. If the terminal is operating in a restricted access mode, the *Proxy Only in Kidsafe* emergency mode is enabled. In this mode, only kidsafe users engage the proxy service.

If the WebTV service is unavailable and a user attempts to connect through a POP, he or she is directed to the Star service for further information.

Chapter 2

Service Files

The WebTV services use three types of files to control and log a service's behavior. This chapter provides an overview of the structure and syntax of each of these files. See the *WebTV Service Operations Manual* for details on changing or extracting information from these files.

This chapter contains the following sections:

- **Configuration File**—describes arcadia.cf, the configuration file that resides on each server running the WebTV services.
- **HTML Template Files**—describes the structure and syntax that make up the WebTV template files.
- **Service Log Files**—provides an overview of each log file used by the WebTV service.

Configuration File

Each server that runs a service or set of services contains a configuration file that tells the service how to behave, where to store files, what level to log, and so on. The configuration file is created from a master configuration file called arcadia.cf. This file contains a list of standard C language ifdefs and macros that expand to values or path names for a particular service; arcadia.cf also contains options that can be set to specific values for a service. As part of the build process, arcadia.cf is run through a C preprocessor and creates a number of configuration files for all of the supported services. All of these files are placed on each server in `production/webtv/config`. When the installation script is run to install the services, a file, called *webtv.solaris2.cf*, is created in the same directory. This file is actually a symbolic link to one of the configuration files created from the build process; for example, for a PRODUCTION network, a symbolic link is made to *production.solaris2.cf*. The current list of possible configurations includes:

Table 2-1 Service Configuration Files

Service Network	Configuration File	Used For
PERSONAL	personal.solaris2.cf	Development
TESTDRIVE	test.solaris2.cf	Release Testing or Daily Builds
THRASH	thrash.solaris2.cf	Load Testing
DEMO	demo.solaris2.cf	External Demos
JAPAN1	japan1.solaris2.cf	Japan Trial 1
PRODUCTION	production.solaris2.cf	Customer Release

File Organization

The configuration file is made up of three parts: global settings, service settings, and test settings.

Global Settings

The global settings appear first in the file and set either global options or options for more than one service. Currently, the global portion of the configuration file contains:

- ifdefs that specify which service to build
- options that specify
  - which FlashROM version to download to the terminal
  - how large log files can become and where to store them
  - what level of detail to record in the log files
  - which PhoneDB to base the tellyscript generation on
  - a definition of each service
  - a definition of who belongs to which service group

Service Settings

The service settings appear second in the file and specify options specific to each service. Because the file is executed top to bottom, the options in any particular service can overwrite global options. Each service can have a number of options to specify how it behaves. The possible options are:

- name—The name of the service. Normally appears as the first part of a URL. Services that WebTV invented have a “wtv-” prefix.
- host—The host name of the machine(s) the service resides on.
- exe—The name of the binary that the service launcher should run to start this service.
- port—The TCP port number the service uses.
- nostart—If this is set to true, the service launcher will not try to start the service or will start the service in other ways; for example, to start duplicate mailto and wtv-mail entries leading to the same service. This option is useful for services that need debugging.
- arg0—Used to change the value of argv[0] that the process is given. This option is used to look up the port number in the Services section, and it is also

used to find the config section later in this file. This option is handy if you want to run two copies of the same binary and use different options.

- flag—Sets a flag for the service, which will be sent to the terminal when the headwaiter advertises the service. More than one "flag=" attribute can be specified; they will be ORed together. You can specify the flag as a hexadecimal number by starting it with "0x", or you can specify it by name. The currently known names are:

  DontEncryptRequests (Do not encrypt requests from client -> server)

  UseHTTP (Instead of WTVP)

  WideOpen (Allow untrusted links to this service)
- noAdvertise—Can be set to true to indicate that this service should never be advertised to terminals.
- NoHelp—Used for services that don't have a "help" command; for example monitord uses this when monitoring services.

Test Settings

The test settings appear last and include all of the options necessary to specify the tests that need to be run to check if a service is operational.

Syntax

Syntax for configuration files includes normal C syntax ifdefs, macros, as well as four WebTV-specific syntaxes, listed in Table 2-2.

Table 2-2 WebTV-Specific Configuration File Syntax

Syntax	Description	Example

<service> </service>	Indicates the beginning and end of a specific service configuration.	`<harvester>`
		`Logfile ARCADIA_LOG(harvest-er.log)`
		`</harvester>`

@	Comment line used to include comments about the log file. This comment line will not be included in the preprocessed configuration file.	`@Here is a definition of each type of service.`
;	Used to include comments about the log file. This comment line will be included in the preprocessed configuration file.	`; Service configuration --`
\	Continues a line.	`name=wtv-guide\` `host= ## hostname\`

HTML Template Files

WebTV servers use template files to produce HTML pages to send to the Internet terminal. Templates generally reside in the content directories for the running servers and contain HTML and proprietary tags that create the user interface for pages that are displayed by the Internet terminal.

Template files are organized into labelled sections and include directives that allow for customization of the pages on a per-request basis. Each labelled section is an HTML page or portion of a page. Labelled sections can also be included by other sections. These directives include the ability to insert or substitute text into the page that is generated by the service, include other templates and labelled areas, and conditionally include sections of templates.

Templates might be changed to fix the user interface or to change the display that a user might see. They provide the bulk of the text and formatting that is on the screen. For example, an operations person may change the templates in the stard service to provide a different warning message when the service is unavailable.

Template files are also used by the setupd and registerd servers to express the mail messages that are sent to new users when their accounts are created. These templates have substitutions for email addresses and the user's names to customize the message and are in standard UNIX mbox format.

Template Syntax

The utility library contains a set of routines to use template files to generate HTML code for the clients programmatically. During the process of writing a template file to the output, `WriteTemplate()` can do substitutions for parameters (passed in as name/value pairs as a string dictionary), as well as execute conditional directives in the template file. The syntax of the template files, used in addition to the "standard" HTML tags, is described below.

{}

{*parameterName*}

Purpose: Substitutes the value of the specified *parameterName* into the page.

Options: *parameterName* can include the following options:

- `%` — The data will be escaped for use in a URL with the %xx format. The prefix `%` escapes everything except the characters: a-z, A-Z, 0-9, `$ _ - . ! * ' ( )`.
- `&` — The data will be escaped for use in HTML. The prefix `&` escapes include:
  - `< - <`
  - `> - >`
  - `& - &`
  - `" - "`
  - `tab -      `
  - `+ -` Same as `&` escaping but also makes spaces nonbreaking.
  - `space -  `

Example : For example if *parameterName* is the value foo and foo is "hello world", then {%foo} would be replaced with "hello%20world." Likewise, if *parameterName* is the value foo, and foo is "Mom & Dad, the {&foo} would be Mom & Dad."

LABEL

<LABEL *labelName*>... </LABEL>

Purpose: Describes a labelled area within the template file. The caller specifies this label when calling `WriteTemplate()`. Only the text within thislabelled area is omitted.

Example :

```
<LABEL happymessage>
We are all happy.
</LABEL>
```

INCLUDE

<INCLUDE *labelName*>

Purpose: Causes the contents of the section `labelName` to be omitted at this point. Recursive `INCLUDE` directives are not supported.

Example : `<Include happy message>`

IF or ELSE

<IF *parameterName*="*value*"> or:
<IF {*parameterName*}="*value*"> or
<IF {*parameterName*}={*otherParameter*}>
...
<ELSE>
...
</IF>

Purpose: Omits the text between the `IF ELSE` syntax. If the value of *parameterName* is equal to the value within quotes or the value of the *parameterName* within braces, then the text before the `<ELSE>` directive will be omitted. If it is not equal or the parameter is not defined, the text between `<ELSE>` and `</IF>` will be omitted. <IF *parameterName*!="*value*"> is also supported.

To compare values as numbers, put a # before the operator. Supported operations are: `#=` `#!=` `#>` `#<` `#>=` `#<=`.

IFDEF

<IFDEF *parameterName*>
...
<ELSE>
...
</IFDEF>

Purpose: Omits the specified directive. If *parameterName* has been defined, the text before the `<ELSE>` directive will be omitted. Otherwise, the text between `<ELSE>` and `</IFDEF>` will be omitted.

Example :

```
<IFDEF day>
   Today is {day}
<ELSE>
   I don’t know what day it is.
</IFDEF>
```

IFNDEF

```
<IFNDEF parameterName>
...
<ELSE>
...
</IFNDEF>
```

Purpose: Omits the specified directive. If the parameter `parameterName` has not been defined, then the text before the `<ELSE>` directive will be omitted. Otherwise, the text between `<ELSE>` and `</INDEF>` will be omitted.

Example :

```
<IFNDEF I don't know what day it is.>
<ELSE>
     Today is {day}
</IFNDEF>
```

DEFINE

```
<DEFINE parameterName="value">
<DEFINE parameterName={parameter2Name}>
```

Purpose: Adds the parameter *parameterName* with the specified *value*, or the value of *parameter2Name* to the current dictionary for this template.

Note: This parameter is now permanently entered in the dictionary.

Example :

```
<Define country="France">
Welcome to {country}
```

FILE

```
<FILE filename.tmpl>
```

Purpose: Adds the specified file to the template. Use this syntax to include `LABEL`s from another file that are used elsewhere. The `<FILE>` command works outside of other template directives; that is, you cannot conditionally add other templates.

Example : `<FILE filename.tmp>`

Note: Multiple white space and comments in the template will not be omitted (though parameter values that contain either will).

An Example Template File

The following example uses `WriteTemplate()` to sends the user the label "mainpage." The following variables were defined beforehand:

```
user-name = Dean
```

```
user-has-friends = false
```

```
user-friend-count = 1
```

In addition, a label in a "shared" file was created:

```
<LABEL signature>
        Sincerely,<p>
```

```
        WebTV Customer Care
</LABEL>
```

The final HTML output would look similar to the following:

```
<HTML>
<HEAD>
<TITLE>Fun For Dean</TITLE>
</HEAD>
<BODY>
Hey Dean,<p>
Why don't you call your mother.
It's good to have a friend.

Sincerely,<p>
WebTV Customer Care
</BODY>
</HTML>
```

Service Log Files

Each service has at least one log file associated with it that typically records the entire connection process of a terminal. To troubleshoot a given customer's problems, you can sift through the logs by searching for the terminal's serial number and find out where a particular problem (usually connection) is occurring.

Directory Structure

The WebTV Service records a variety of logs, including the service logs in the directory:

`~production/webtv/log`

Additionally, on each production machine a file called *user-events.log* in the directory

`~production/webtv/log/event_log`

details all of the various transactions a given machine executes in the course of providing the service. Each entry is labelled to indicate the time and the process involved, and the entry records data on the transaction. These entries are saved for long term record, whereas the service logs are usually kept for five days.

Major Log Files

Briefly described below are the major service logs recorded by the WebTV Service.

mailtod.log—Found on the mailtod machines, it records the events pertaining to mail, including how mailtod is handling MIME attachments, HTML-based messages, and the like.

logserverd.log—Found on the storeless machines, it records the phone logs from the client, including data on any disconnections or failed connections since the last time a phone log was recorded for a given client, as well as data on successful connections.

proxyd.log—Found on the proxy machines, it records the events associated with serving HTTP requests, transcoding pages, and page caching.

preregistrationd.log—Found on the preregistration machines, it records the events associated with a client being run through the preregistration process. These events include handing out tellyscripts and recording client ANI information from the preregistration dial in service.

headwaiterd.log—Found on the storefull machines, it records the comparisons between the tellyscripts the terminal currently has and what the current tellyscript should be. It records when a new tellyscript is given to the terminal, and also records giving services to the clients as they connect to the WebTV service.

Logs are also kept for the smaller services and include the following (many of these are logs that are recorded on more than one, and frequently all, production machines):

storefull machines—*cookied.log*, *explored.log*, *favorited.log*, *flashromd.log*, *harvestd.log*, *registerd.log*, *servicelauncher.log*, *setupd.log*, *statusd.log*

storeless machines—*aroundtownd.log*, *channeld.log*, *contentd.log*, *crashrecord.log*, *flashromd.log*, *guided.log*, *harvestd.log*, *homed.log*, *searchd.log*, *servicelauncher.log*, *statusd.log*, *tricksd.log*

prereg machines—*anid.log*, *harvestd.log*, *servicelauncher.log*, *statusd.log*

mailtod machines—*aliasd.log*, *flashromd.log*, *harvestd.log*, *servicelauncher.log*, *statusd.log*

mailsorter machines—*aliasd.log*, *harvestd.log*, *servicelauncher.log*, *statusd.log*

postoffice machines—*aliasd.log*, *harvestd.log*, *mail.local.webtv.log*, *servicelauncher.log*, *statusd.log*

Typical Log Problems

The *logserverd.log* and *preregistrationd.log* are the most useful logs for tracing connection problems. One of the first troubleshooting tactics for connection problems is to force the terminal to go through the preregistration cycle again. The *preregistrationd.log* records the ANI received by the server and the details of the tellyscript subsequently sent down to the terminal.

When there are entries in the *preregistrationd.log* and no corresponding entries in the *logserverd.log*, it demonstrates that the terminal is functional, but that there is a breakdown between either the client and the local POP, or between the local POP and the WebTV service. This breakdown may be the result of a bad dialing pattern, the necessity of an access code for possible toll calls, or a variety of other causes.

Log File Format

Most of the WebTV service logs share the following three-line format:

```
<error-message>: <error-type>

<reporting-function>()            <client-id>/<subscriber-id>
<reporting-file>:<line-number>    <service>[<pid>]              <date> <time><flag>
```

Parts of the Log File

A log file is made up of the following parts:

Error message—The error message is an approximate description of what went wrong. It should always appear first in a log entry.

Error type—The error type is a refinement of what went wrong, when the system has a specific idea of where it failed. Unexpected EOF and broken pipe are common sorts of messages to see here. Error type does not appear in all log entries.

Reporting function—The reporting function is the section of the mail service that found the error. It will always appear in a log entry.

Client id—The client id is a seven-digit number that refers to the terminal itself specifically and uniquely. -/- indicates that this information is still unknown. This could be simply because the terminal hasn't been registered yet, or more likely, at this point in the connection process, the service hasn't received this information from the terminal.

Note: There is also a user id, which relates specifically to one of the (potentially multiple) users on a terminal, but this information does not show up in the log.

Subscriber id—The subscriber id is a hexadecimal string linked to a single name and billing address. `-/-` indicates that this information is still unknown, for the same reasons the client id wouldn't be known.

Reporting file and line number—The reporting file is the specific file that found the error and reported it, and the number that follows refers to the line of code that encountered the error.

Service—The service refered to in the log files.

Process id (pid)—The process id is the number that refers to the process that encountered the error. You can use the pid in conjunction with the timestamp to determine which of multiple log entries refer to the same event.

Date and time—The date and time are a stamp to isolate when the event actually happened. Used in combination with the process id, it can isolate which of multiple entries refer to the same problem.

Flag—The flag is an asterisk that can appear after the date and time. Currently, the system tries to delineate between serious problems (errors) and warnings or irregularities. The presence of the flag does not guarantee that the problem is serious but indicates it should be looked at further. The lack of a flag does **not** indicate that the problem isn't serious, only that the system hasn't been able to interpret it as such.

An Example Log File

The following example shows a log entry.

```
Error reading headers from client: Unexpected EOF
HandleCommands()        -/-
Protocol.c:203          mailtod[235]            10/03 12:46:22*
```

Where:

- `Error reading headers from client:` is the errror message
- `Unexpected EOF` is the error type
- `HandleCommands()` is the reporting function
- `-/-` is the client id and the subscriber id
- `Protocol.c:203` is the reporting file and line number
- `mailtod [235]` is the service
- `10/03 12:46:22` is the date and time
- `*` is the flag that notes the problem should be looked at

Note: It's fairly typical to have more than one log entry for a given error. Duplicate entries occur as the error codes percolate up the stack.

Chapter 3

WTVP Protocol

The WebTV service uses a modified version of HTTP called WebTV Protocol (WTVP). HTTP requests are converted to WTVP for optimal speed on the Internet Terminal.

This chapter describes the WebTV protocol and discusses security offered by WTVP. It contains the following sections:

- **Components of WTVP**—describes the differences between HTTP 1.0 and WTVP.
- **WTVP Headers**—describes the MIME headers specific to WTVP.
- **WTVP Security**—describes the special header that creates the security encryption layer between the terminal and service and provides pointers to debugging WTVP encryption-caused problems.

Components of WTVP

WTVP is identical to HTTP 1.0 except that it has the following additional features.

- **Persistent Connections**—When a terminal issues an WTVP request to a server, the connection is held open unless the terminal needs to connect to a server with a higher priority.
- **Data Encryption**—Encryption of data is supported at the level of the body. When the terminal sends a request, the server issues a response telling the terminal that the body is encrypted.
- **New Headers**—New headers have been added to WTVP that are not in HTTP. A slightly different format for the headers is also used. All of the headers are of the form of `wtvp-` rather than `x-`, for example:

  WTVP: `wtv-card-number` HTTP: `x-xbitmap`
- **Compression**—WebTV supports its own compression algorithms. WTVP uses transcoding to convert Web content into formats the service can easily handle. For example, GIFs are translated into JPEGs, and images are downsized or scaled to fit the television screen. WTVP also rewrites HTML into formats that the WebTV system can easily handle. A proprietary compression algorithm called LZPF is also used to further compress the HTML.

WTVP Headers

The WebTV Internet terminal uses standard MIME headers and headers that are unique to the WebTV network.

Standard HTTP Headers

Table 3-1 describes headers that originate in WebTV services; they are sent to the terminal when a service responds to a request for data.

Table 3-1 Standard HTTP Headers

Header	Description
`accept: %s`	List of accepted data types. This header is sent when requesting data from a service.
`content-length: %d`	The number of bytes in the body of a transaction. This attribute is sent by the terminal and by services when sending data.
`server: %d`	The server ID and version string. The terminal keeps this information as a debugging aid.
`user-agent: %s`	The terminal ID and version string. This header is sent when a terminal sends requests to a service.

WTVP Attributes

Table 3-2 describes headers that originate from the terminal; they are sent to WebTV services during the initial request to a particular service.

Table 3-2 WTVP Attributes Originating from the Terminal

Header	Description
`wtv-card-number: %s`	The number on the smart card currently inserted into the box.
`wtv-client-serial-number: %s`	The serial number of the terminal or client software.

Table 3-3 describes headers that originate from the WebTV services; they are sent to the terminal when responding to requests.

Table 3-3 WTVP Attributes Originating from the Service

Header	Description
`wtv-boot-url: %s`	The URL to execute when powering on.
`wtv-bypass-proxy: true|false`	Allows the terminal to bypass the proxy service and go directly to the site, if necessary. This header is on by default and should be turned off for KidSafe mode.
`wtv-client-date: %s`	The current date and time in rfc822 format. For example, Sunday, 07-Jan-96 21:46:45 GMT.
`wtv-client-time-zone: %s`	The name and offset of the terminal's time zone. For example, PST -0800.
`wtv-disconnect-url: %s`	The URL to retrieve just before disconnecting from the WebTV network. The user may reconnect later without logging into the network.
`wtv-expire: %s`	Causes the resource with the specified URL to close. It will be reloaded from the network the next time it is accessed.
`wtv-expire-all: %s`	Causes the resources whose name starts with the specified URL to expire. The connections are reloaded from the network the next time they are accessed.
`wtv-field-name: %s`	The name of the field whose value caused the service to respond with an error message. The terminal will select the field when the error message is displayed.
`wtv-home-url: %s`	The URL of the user's home page. This URL will be stored on the terminal and used whenever the user presses the Home button.
`wtv-inactive-timeout: %d`	The number of seconds before hanging up the telephone due to network inactivity.

Table 3-3 WTVP Attributes Originating from the Service

Header	Description
`wtv-location: %s`	A redirection with data. This is similar to HTTP's `location: redirection` mechanism, but the data is also sent at the same time.
`wtv-logo-duration: %d`	The number of seconds the logo is to remain visible on the screen.
`wtv-logo-interval: %d`	The number of seconds to delay before redisplaying the logo.
`wtv-logout-url: %s`	The URL to retrieve just before logging out from the network because the user pressed PowerOff.
`wtv-mail-count: %d`	The number of new messages waiting in the user's mail box. This information will be used by the terminal to control the message indicator on the front of the hardware.
`wtv-name-server: %d.%d.%d.%d`	Sets the IP address of the name server.
`wtv-parser-ignore: %s %s`	Directs the HTML parser to ignore all HTML tags and attributes between the two specified tags.
`wtv-reconnect-url: %s`	The URL to get just after reconnecting to the network.
`wtv-search-url: %s %s`	The URL of the search page. This URL is stored on the terminal and used whenever the user presses the Search button on the remote control (when available).
`wtv-service: name=%s host=%s port=%d flags=%x`	A WebTV service description. The flags are as follows: 0x00000001 `kServiceEncryptRequests` 0x00000002 `kServiceUseHTTP`
`wtv-service: reset`	Directs the terminal to reset its list of services.
`wtv-service-timeout: %d`	The number of seconds to wait before attempting to contact a service after a connectionattempt failed.
`wtv-trusted: true | false`	Treats the data as untrusted; no matter if the value is true or false.
`wtv-visit: %s`	A URL to visit after the current document is received and displayed to the user (if not null). These URLs will be added to a list and visited in the order received.

WTVP Security

In addition to the general WTVP encryption, there is a particular security header called `wtv-ticket` (see Table 3-4). The Headwaiter initially gives this header to the terminal, which in turn sends it to each service that it wants to access. The header primarily contains all of the information necessary to encrypt data that is sent to a service, and it can only be decrypted by a service.

WTVP Server-signed Headers

Table 3-3 describes a header that originates in WebTV services. It is sent to the terminal when responding to requests. The header is returned to the WebTV services during the initial request to a particular service. The headers that are encoded into the `wtv-ticket` header use the private key of a WebTV service. They are not used by the terminal, but are accessible to network services and applications.

Table 3-4 Encryption Header with Encoded Headers

Header	Description
`wtv-ticket:%s`	
Headers Encoded in `wtv-ticket`	
`wtv-user-id: %s`	The current user's ID.
`wtv-user-name: %s`	The current user's name. This is the email name of a person who is authorized to use the WebTV account.

Debugging Encryption-Caused Problems

If the terminal tries to display what it thinks is good HTML, but garbage is instead appearing on the television's screen, it probably means that the terminal and service got out of synchronization in the encryption process. To troubleshoot this problem, you can turn off the encryption for a particular service that you are having problems with by starting the service with `allow_encryption` set to `present`. You can also turn off the encryption for an individual terminal if you have a debugging terminal. Typing 93288 brings up a connection to a special setup that allows you to turn off encryption.

Chapter 4

Access Services

Access services include services that handle preregistering users, registering users, validating and greeting users, notifying users of problems, and logging user's movements. This chapter described each service by providing a description of the components, operation, and special files that it uses. It contains the following sections:

- **Headwaiter Service**
- **Star Service**

Headwaiter Service

The Headwaiter (headwaiterd) service is central to the WebTV network. headwaiterd handles logging users into the WebTV network every time they turn on the Internet terminal. headwaiterd greets terminals wanting to enter the service, authenticates them, and introduces them to the other services.

This section describes the following components:

- **Headwaiter Service Components**—describes the components that make up the Headwaiter system.
- **Headwaiter Operation**—describes the operation of the Headwaiter system.
- **Special Files**—describes the log files produced by headwaiterd, and the configuration file and templates used by headwaiterd.

Headwaiter Service Components

Figure 4-1 illustrates the Headwaiter system. The parts of the Headwaiter system are described following the figure.

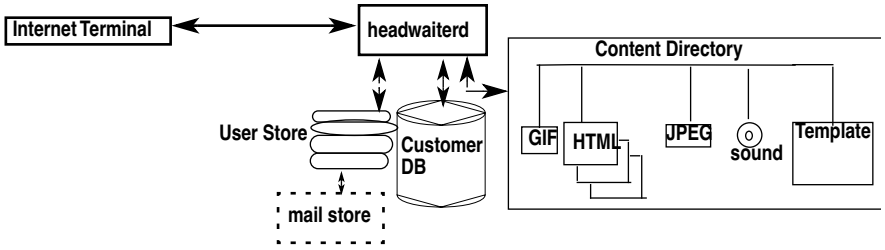


Figure 4-1 Headwaiter System

headwaiterd—A program that authenticates a user and serves them the list of available services.

Customer Database—An Oracle database used by headwaiterd to access customer account information.

User store—The user store is a program that resides on a NFS (Network File System) storage server, and contains user specific information for several servers. headwaiterd accesses the mail user store to see if there are any new messages for the user, and if so, notifies the Internet terminal.

Content Directory and Templates—The content directory contains standard html files, which can include gifs, jpegs and various sound formats. The content directory can also contain template files, which have .tmpl suffixes. Templates contain the formatted HTML content, and variables that determine the appropriate content to display. contentd serves ordinary HTML files and executes template pre-processing directives.

Headwaiter Operation

Figure 4-2 illustrates the headwaiterd sign-in process. headwaiterd's operation is described in detail following the figure.

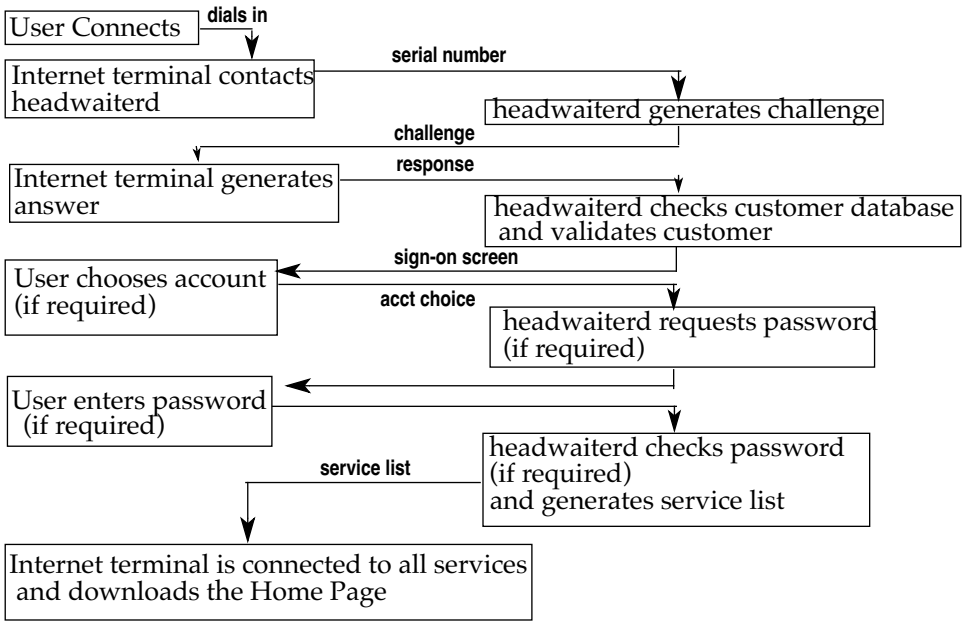


Figure 4-2 Headwaiter sign-in process

When the Internet terminal calls the WebTV network, headwaiterd initiates a challenge and response sequence to identify the Internet terminal. The terminal responds to the challenge by decoding an encryption string, using a key downloaded during preregistration.

If the Internet terminal is successful, headwaiterd verifies whether the subscriber's account is in order, by checking the customer database.

If this is a first time connection to the WebTV network, the subscriber is referred to the registration server.

Once the subscriber's account is validated, headwaiterd checks the customer database to determine if there are multiple users on the account. If it is a multi-user account, headwaiterd prompts for a user name. If the account requires a password, the user is prompted for the password.

If it is a single-user account, the account is displayed automatically; if the user has configured the account to require a password, headwaiterd prompts for a password.

After validating the user, headwaiterd advertises a list of named services to the Internet terminal. Named services can include, for example, http: or wtv-mail:. The Internet terminal also downloads a list of configuration parameters, such as time-outs, relevant URL's, and redundant named services.

Additionally, when a new subscriber contacts headwaiterd for the first time, the server provides the Internet terminal with the address of the stard. stard is the default service used when the WebTV network cannot be reached. This information is stored by the Internet terminal, and used when required.

Headwaiter is also the service the terminal connects to when doing message watch. The user can use the Setup service to configure the Internet terminal, for example, to dial in at 7 a.m. and check mail.

The Internet terminal dials in at the specified time and connects immediately to headwaiterd. headwaiterd checks mail store, and turns on the Internet terminal light if there are messages.

In addition to signing in an Internet terminal, headwaiterd performs the following two special functions:

- headwaiterd determines if an Internet terminal meets the current minimum standards and whether an upgrade is available. If the Internet terminal's software is out of date, headwaiterd forces the terminal to visit flashromd to download an updated version of the software.
- If a user's account is delinquent, headwaiterd prevents the Internet terminal from logging on, and requests that the user correct the problem by either providing a new credit card number or by calling WebTV's 800 number.

Special Files

headwaiterd produces log files and uses the configuration file to control the operation of the service.

Error Log

headwaiterd.log is used by headwaiterd to record errors, warnings, or other more or less anomylous notifications. It is stored in `~production/webtv/log/`.

See Appendix B in the *Service Operations Reference* for detailed information on error log files.

Events Log

user-events.log is used by all services, including headwaiterd, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/user-events.log`.

Note: The event log is a more complex process than error logging in that it is necessary for different programs to communicate with each other to log events.

See Appendix B in the *Service Operations Reference* for detailed information on events log files.

Configuration files

headwaiterd downloads nameserver and client parameters (including service list), as headers, to the Internet terminal.

The portion of the arcadia.cf file that controls headwaiterd is configured with many options. The areas of interest to an operations person are described below.

The following options can be set by an operations person:

- **max user count**—is the maximum number of users for an Internet terminal. max user count points to the phone database containing the tellyscripts. (telly scripts tell the Internet terminal what phone number to call, how to dial, and what kind of local pop to connect to.)
- **http timout**—length of time headwaiterd should wait while trying to contact notifyd. HTTP timeout is currently set to five seconds.

Star Service

The Star service is a default service that serves an HTML page to the Internet terminal when a requested service is either down for maintenance or malfunctioning. The star service also serves an HTML page to the Internet terminal when a requested URL is invalid.

The Star service typically runs on its own machine and is independent from the rest of the services.

This section describes the following components:

- **Star System Components**—describes the components that make up the Star system.
- **Star Operation**—describes the operation of the Star system.
- **Special Files**—describes the log files produced by stard, and the configuration file and templates used by stard.

Star System Components

Figure 4-3 illustrates the Star system. The parts of the system are described following the figure.

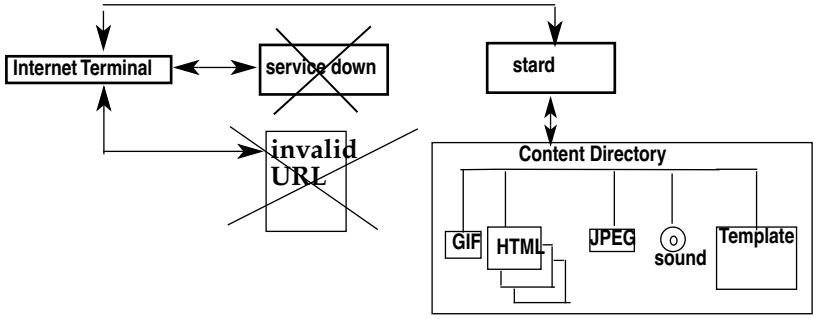


Figure 4-3 Star Service

stard—Stard is a program that serves content to an Internet terminal when a service is either down for maintenance or malfunctioning. Stard also serves content to an Internet terminal requesting an invalid URL.

Service Down—When a service is either down for maintenance or malfunctioning it does not respond to the Internet terminal.

Invalid URL—When a requested URL is invalid it does not respond to the Internet terminal.

Content Directory and Templates—The content directory contains standard HTML files, which can include gifs, jpegs and various sound formats. The content directory can also contain template files, which have .tmpl suffixes. Templates contain the formatted HTML content, and variables that determine the appropriate content to display. contentd serves ordinary HTML files and executes template pre-processing directives.

Star Operation

The Star service has the following three main uses:

- Serving an HTML page to the Internet terminal when a requested service is down for maintenance.
- Serving an HTML page to the Internet terminal when a requested service is malfunctioning.
- Serving an HTML page to the Internet terminal when no service is available for a requested URL (usually because of an invalid URL).

When an Internet terminal fails to connect to a requested service, it refers to its service list to determine if there is a secondary server to use for connection. .

If the Internet terminal fails to connect through a secondary server, it defaults to stard (which is independent of the other services and is always available). stard then serves an HTML page to the Internet terminal; for example, "The service is down for maintenance, please try again later."

When an Internet terminal fails to connect to a requested URL, for example, if the URL does not contain http, it will default to stard. stard then serves an HTML page to the Internet terminal; for example, "WebTV can not handle this type of information."

Special Files

The Star service produces log files to record information, uses configuration files to control the operation of the service, and uses templates to display content.

Error Log

stard.log is used by stard to record errors, warnings, or other more or less anomalous notifications. It is stored in `~production/webtv/log/stard.log`.

See Appendix B in the *Service Operations Reference* for detailed information on error log files.

Events Log

user-events.log is used by all services, including stard, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/user-events.log`.

Note: The event log is a more complex process than error logging in that it is necessary for different programs to communicate with each other to log events.

See Appendix B in the *Service Operations Reference* for detailed information on events log files.

Configuration file

The following sample shows the section of arcadia.cf that controls stard. There are no options that need to be set or changed by an operations person:

```
<stard>
ServeProcessCount 20
MaxDescriptors 100

ContentRoot /usr/home/production/webtv/content/stard
TemplatesRoot /usr/home/production/webtv/content/stard/templates
LogFile /usr/home/production/webtv/log/stard.log
ContentRoot /usr/home/production/webtv/content/stard
</stard>
```

Templates

stard uses a template called *unavailable.tmpl*, which contains the formatted HTML content and variables that determine the appropriate content to display. *unavailable.tmpl* contains the page that is served when an Internet terminal defaults to stard.

unavailable.tmpl can be replaced by an operations person by renaming it after one of the following three templates:

- Daily *maintenance.tmpl*
- Weekly *maintenance.tmpl*
- Generic *unavailable.tmpl*

The following samples can be modified by an operations person and used when replacing unavailable.tmpl:

- Daily maintenance—"We're closed for maintenance tonight and will be available at <variable>."
- Weekly maintenance—"Every <variable- day> we'll be unavailable from <variable-time> to <variable-time>."
- Generic unavailable—"The WebTV network is temporarily unavailable, please try connecting later."

Chapter 5

Messaging Services

Messaging services include services that handle all forms of messaging, including sending and receiving mail and reading and posting to news groups. This chapter documents each service by providing a description of the components, operation, and special files that it uses. It contains the following sections:

- **Mail System**

Mail System

The mail system is composed of the mailtod service, which is a program that handles most mail actions, and a set of mail-specific programs that work with other WebTV software services to handle all of the actions necessary to send and retrieve email. The set of programs and services are collectively called the *mail system*. This section describes these components in detail:

- **Mail System Components**—describes that components that make up the mail system.
- **Mail Operation**—describes the various operations that the Mail service handles (listing a mailbox, reading, moving, deleting, sending, forwarding, or replying to a message, and managing the users' address book).
- **Special Files**—describes the log files, configuration file, and templates that the Mail system uses.
- **Software Tools**—describes the software tools that the mail system uses.

Mail System Components

The mail system service is illustrated in Figure 5-1. The parts of the mail system are described following the figure.

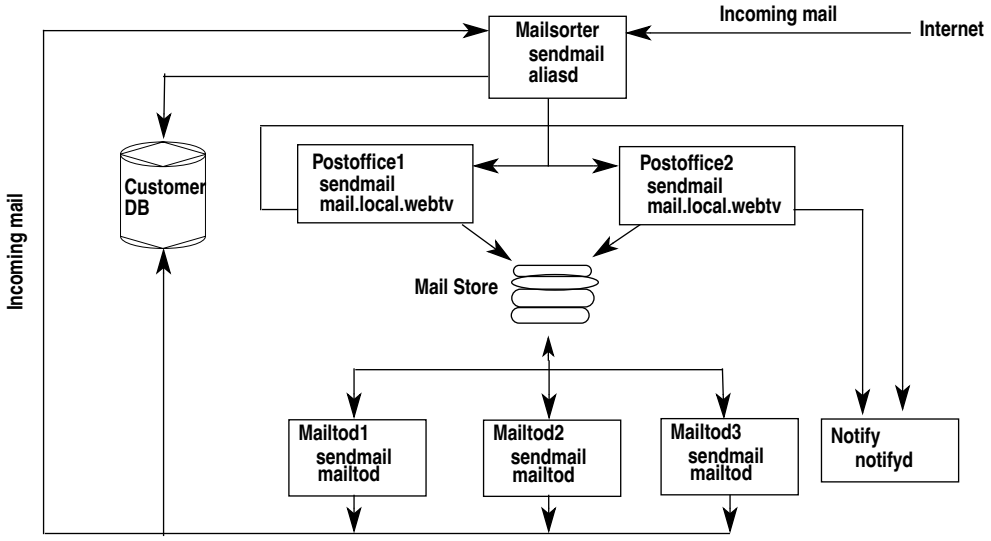


Figure 5-1 Mail System

mailsorter server

The mailsorter receives email messages from the Internet and forwards them to the appropriate postoffice; it also receives messages from the mailtods and forwards them to the Internet. It runs sendmail and aliasd.

- **sendmail**—a standard UNIX mail transfer agent (MTA). An MTA is a program that forwards mail between hosts, but it doesn't perform the actual delivery or retrieval of the mail.
- **aliasd**—an internal service used by sendmail to determine which postoffice should receive an inbound message for a WebTV user. For example, suppose that there are two service centers in operation, one in the United States and one in Japan. When a mailsorter (in the United States or Japan) receives a message meant for a particular user, it will ask aliasd where the user's mail is stored. If it is stored in Japan, the message is forwarded to the Japanese postoffice; if it is stored in the United States, the message is stored in the U.S. postoffice.

postoffice Server

The postoffice receives messages from the mailsorters, stores them into the users' mailboxes, and alerts the notify service when new mail arrives. It runs sendmail and mail.local.webtv.

- **mail.local.webtv**—a program that opens a user's mailbox and delivers mail to it. It also alerts notifyd that mail has arrived.

Mail Store Server

The mail store resides on a NFS (Network File System) storage server (NetAPP), and contains user specific information for several servers. The mail store is used by the mail.local.webtv and mailtod to store and retrieve messages.

notify Server

The notify server runs notifyd.

- **notifyd**—a program that turns on the red light on the terminal when a postoffice tells the notify service that a user has new mail.

mailtod Server

The mailtod server is the machine that communicates with the Internet terminal and enables email-related actions on behalf of users. It runs sendmail and mailtod.

- **mailtod**—the service that a user's Internet terminal communicates with. mailtod acts on behalf of WebTV users and lists the content of their mailboxes, retrieves their messages, moves messages between mailboxes, and sends messages.

Customer Database

The customer database is an Oracle database used by aliasd and mailtod to find out where a user's mail is stored.

Note: The number of machines running the mail service varies; that is, there can be any number of machines dedicated as mailsorters, postoffices, and mailtods. However there is only one NetAPP per service group and only one customer database for all users.

Mail Operation

The mail system functions as the interface between the UNIX mail-handling application sendmail and the Internet terminal. The mail service handles the following types of operations: accepting and delivering mail, listing a mailbox, reading, moving, deleting, sending, forwarding, or replying to a message, and managing the users' address book. Each of these operations is discussed in the following subsections.

Receiving Mail

An incoming message is received by sendmail running on a mailsorter. sendmail connects to aliasd and gives it the username(s) of the recipient(s). aliasd returns the recipients's user ID, subscriber ID, and user category together with the name of the appropriate postoffice. The message is next received by a postoffice's sendmail and delivered into a user's mailbox, using the mail.local.web. notifyd makes a note of this and tells the user (using a red light and a beep) that she or he has new mail. If the user is not connected at the moment, the next time that user connects, notifyd will turn on the red light.

Listing and Reading Mail

When the user wants to read his or her mail, the Internet terminal connects to the first available service by sending a URL to a mailtod, asking it to list the received mail. mailtod responds by opening the user's inbox and looking inside at

messages that were received. mailtod then composes a page of HTML and sends it back to the terminal. The user clicks on a message link, and mailtod opens the mail, formats it in HTML, and sends it back to the user.

Moving or Deleting Mail

When a user finishes reading a message they may choose to delete or move the mail message to another mail box. When this occurs mailtod accesses mail store to move or delete the message.

Sending, Forwarding, or Answering Mail

When the user sends or responds to a message, mailtod is notified and returns a message form in HTML. When the user submits the form, mailtod extracts the information from the message and turns it into a format that outside mailers will be able to read. It then hands the message to sendmail; sendmail sends it to the mailsorter. The mailsorter will either send the message out to the Internet or will reroute it to another WebTV user.

Managing an Address Book

A user can create an address book with a list of email addresses that they can use to address email. The mailtod handles setting up the address book by creating a file on the mail store and storing the email addresses. When the user views the addresses, mailtod looks at the current addresses in the mail store and creates an HTML document to display them.

Special Files

The mail service uses log files and configuration files to log and control the operation of the service.

Log Files

The mail service uses three log files to record information about its actions: *syslog*, *mailtodlog*, *eventslog*. See "The WebTV Log File Appendix B" in the *Service Operations Reference* for detailed information on log files.

- *syslog* is used by send mail to store and record each message that passes through the server. It is stored in `/var/log/syslog`.
- *mailtod.log* is used by mailtod to record errors, warnings, or other more or less non-standard notifications depending on the current log severity level. Although errors in general will be marked with an asterisk at the end of the line, there is no canonical list of "safe" vs. "unsafe" messages. It is stored in `~production/webtv/log/`.
- *user-events.log* is used by all services, including mail, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/`.

Configuration files

The portion of the arcadia.cf file that controls mail is configured with several options. An overall option called `MailDomain` sets the default domain for all local WebTV users. For the Production service, this is set to the domain webtv.net.

For each service group, there also may be a line specifying which postoffice mail to users of this group should be sent to. (This is used by aliasd.) For example, the following lines:

```
ServiceGroup AlphaProductionServices alpha gamma beta
       IncludeServiceGroup ProductionServices-Base
       PostOffice postoffice-alpha.alma.webtv.net
       [...]

ServiceGroup ProductionServices *
       IncludeServiceGroup ProductionServices-Base
       PostOffice postoffice.alma.webtv.net
      [...]
```

would send mail to alpha, gamma, and beta users to postoffice-alpha.alma.webtv.net whereas the rest would go to postoffice.alma.webtv.net (the asterisk matches everyone).

aliasd has a `RedirectCategoryToHost` option that takes two arguments, a `category` (user group) and a `domain name`. The effect of this option is to redirect mail to all users in the given category to the given domain or host. For example, this could be used to allow old employee with an outdated address; for example lennart@webtv.net, to be redirected to a new internal mail system at corp.webtv.net. To do this, a set of dummy user records were created all belonging to the same subscriber in the "corp" category. The following line was then added to the configuration file:

```
RedirectCategoryToHost corp corp.webtv.net
```

Software Tools

The mail system does not have any specific tools; however, it uses some of the following generic tools:

- querydb—describes a client, subscriber, and/or user record as seen from the services. This tool takes two arguments: the kind of information to describe, and who to describe it for. The first argument should be one of: -client, -subscriber, -user, -users, or -all. The second argument should be a user name, user ID, subscriber ID, or Internet terminal client ID. These two arguments can be mixed freely; for example, "querydb -client bob" will provide information about Bob's Internet terminal.
- distill-log—presents a normalized event log entry in a more palatable form. Event logs are harvested each night and are turned into a "normalized" form with one line per entry. distill-log takes those entries and present them in a more readable & compact form. To use it, you pass the normalized event log on stdin and read stdout. distill-log has one optional argument -wide which tells it not to cut off the output at column 80.

Chapter 6

Proxy Services

Proxy services include services that handle HTTP, FTP, and gopher protocols as well as maintaining a user's preferences for specific Web sites they may visit. This chapter documents each service by providing a description of the components, operation, and special files that it uses. It contains the following sections:

- **Proxy Service**
- **Cookie Service**

Proxy Service

The proxy server functions as an interface between the Internet terminal and servers on the Web. Its main function is to optimize access to the web for better viewing. The proxy does this by caching Web content when possible and transforming data into more compact forms; for example, GIFs are translated to JPEGs. This section describes how the proxy service works and the files and tools it uses.

- **Proxy Service Operation**—describes the various operations that the proxy service handles.
- **Special Files**—describes the log files and configuration files that the proxy service uses.

Proxy Service Operation

The proxy service uses threepieces of software to facilitate its operations:

- proxyd—service that retrieves content based on user request and caches, transforms, rewrites HTML, restricts content if necessary, and fixes broken HTML with page patch mechanism.
- nsproxy—Netscape Proxy software used to handle ftp and gopher requests.
- nsproxystarter—Program that starts the nsproxy.

The proxy service is illustrated in Figure 6-1.

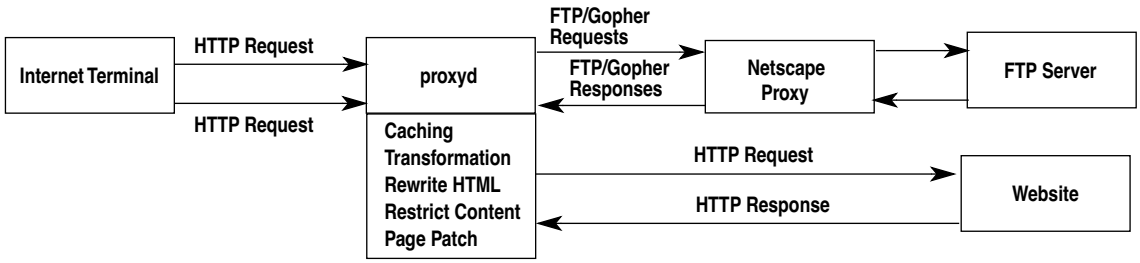


Figure 6-1 Proxy Service

The proxy service handles FTP, Gopher, and HTTP requests. FTP and Gopher requests are handled by Netscape's proxy software; HTTP requests are handled directly by the proxy service.

FTP and Gopher Requests

When a user clicks on an ftp or gopher link, the terminal establishes a connection with proxyd. The proxyd passes the request to another mechanism that is not part of the proxy code itself: nsproxy. The proxy does not directly support ftp and gopher; instead, Netscape's proxy software handles these requests. All of the WebTV proxies recognize ftp and gopher URLs as special, and instead of talking to the actual source server, they forward the request to an available NetScape proxy.

HTTP Requests

When the user clicks on a link to a Website, the terminal establishes a connection with proxyd. This connection is persistent in that it can be used for multiple requests. proxyd receives the connection and determines the following:

- Does the web content need to be transformed?
- Does the web content need to be cached?
- Does the request need HTML rewritten?
- Does the content need to be controlled?
- Does the page need to be patched?

Based on the answer, proxyd hands off the content to a particular subprocess that handles the operation.

Transformations

The proxy service inspects the headers of the Web page and a decision is made as to the type of transformation that should occur. Transformations are then performed to improve the transfer time to deliver content to the terminal. Three types of transformations are done.

- Images are transformed by reducing the detail of a graphic object; for example, saving the object as gif for an icon or jpeg for a picture.

- Unsupported formats are transformed into formats that the terminal understands. For example, there is an encoding method called PNG that the terminal doesn't directly support; it is transformed into jpeg.

Caching

The proxy caches HTTP responses it has received from Web servers on local disk; it can then serve up those responses when subsequent requests for the same page are made. When the proxy receives an HTTP request, it checks to see if an entry has been made for the page. If an entry does not exist, the proxy retrieves the page from the Web server, performs any transformations, caches it (if appropriate), and then sends the page to the terminal. If a cache entry has been made, the proxy checks the time stamp to see if the cache needs to be updated; if it does, it goes to the server and retrieves a new copy. Otherwise, it sends the content to the terminal.

Caching will not occur for the following content:

- Pages that require authentication.
- Pages that have an associated cookie. Exceptions to this rule are certain images that appear to be static data that are in the context of a cookie will be cached.
- Anything that doesn't containlifetime information. The proxy determines how long to keep pages in cache by inspecting headers that are returned by the server. The server typically returns the last modified time or sends an explicit expires tim. If neither of those headers is present, the object isn't cached.

HTML Rewrite

HTML is rewritten to increase the performance as perceived by the user. The biggest gain is created by adding image size hints as the HTML passes through the proxy. An image embedded in HTML without image size hints will frequently force the client to wait until it gets enough of the image to determine its size because its size determines how text flows in the rest of the document.

Content Restriction

If a terminal is operating in kid-friendly or SurfWatch mode, the proxy also serves as a mechanism for restricting content. Currently, two filter sets are used: SurfWatch and kid-friendly. Both of the filter sets are specified in the arcadia.cf configuration file. When a kidsafe request is sent, the proxy service checks the configuration file to see if the content is safe. If it is safe, the content is displayed to the user; otherwise, an error message is sent, saying the request was denied.

Page Patching. The proxy has a page patching mechanism that allows relatively arbitrary page transformations in HTML as it flows through the proxy. The idea behind page patching is that some pages will be very popular but won't render well on the terminal. With page patching, the proxy could be given instructions about how to render the page to make it look better.

Special Files

The proxy service uses log files and configuration files to log and control the operation of the service.

Log Files

Two types of log files record information about proxy server activities.

ns-proxy.log. This file is used by the NetScape proxy to record the events associated with ftp and gopher requests. It is stored in `~production/webtv/log/`

proxyd.log. This file is used by proxy to record the events associated with serving HTTP requests, transcoding pages, and page caching. It is stored in `~production/webtv/log/`

Configuration Files

The portion of the arcadia.cf file that controls the proxy is configured with many options. The areas of interest to an operations person are described below.

surfwatchfiltersdir

The Proxy has an option called surfwatchfiltersdir that specifies the name of the directory in which the proxy will find subdirectories containing kidsafe filters. This option expands to the directory.

```
$HOME/production/webtv/config/surfwatchFilters
```

The proxy expects to find a number of subdirectories, all with the common name format; filters, followed by a keyword that identifies the filters set. For example, blacklist and yahooligans. When the proxy receives a kidsafe request, it applies these filters to make sure the content is safe to display.

Filter Set Error Messages

The configuration file also specifies the error message to return for the particular filter set if a page is not kidsafe.

```
BlockedMessage BlackList "SurfWatch has blocked access to the
requested page."
BlockedMessage Yahooligans "You can only go to Web pages approved by
WebTV."
```

Page Patching

Usually when you have a set of page patches that need to be applied to a Web site, a single file is created that describes all of the patches to apply and the URLs that they are applied to. This file is stored in the configuration directory. arcadia.cf then contains an entry that points the proxy to them. You cannot update the page patch file without restarting the proxy.

nsproxy

The configuration file identifies a service line for the Netscape Proxy. The proxy service looks at the configuration file that contains a table with the NetScape proxy services. It then gives the request to the available nsproxy. The nsproxy formats the page into HTML; the WebTV proxy then treats the content just as it would any other piece of HTML.

nsproxystarter. The nsproxy starter portion of arcadia.cf identifies a configuration template directory file. A set of configuration files that are necessary for nsproxy to operate are stored in this directory.

websim. This server is used by monitord to test the proxy. arcadia.cf contains an entry in the proxy configuration area that specifies the location of websim.

Cookie Service

The Cookie service provides a way to create stateful sessions with Web site requests and responses. cookied allows Internet terminals and Web sites that wish to exchange state information to place Web site requests and responses within a larger context, termed a "session."

This context might be used, for example, in cases where a Web site requires registration information from users before granting them access.

In addition to storing access information, cookied can be used to track a users movements within a site. Or it might be used in one session, for example, in sites that allow shopping, to create a "shopping cart," which consists of a list of a users selections before purchase.

Cookied runs on storefull machines.

This section describes the following components in detail:

- **Cookie Components**—describes the components that make up the Cookie system.
- **Cookie System Operation**—describes the two operations that the Cookie system handles: setting a cookie, and getting a cookie.
- **Special Files**—describes the log files produced by cookied and the configuration file used by cookied.
- **Service Tests**—describes the tests that an operations person can run to see if the the Cookie service is operational.

Cookie System Components

Figure 6-2 illustrates the two types of operations the Cookie system handles: setting a cookie and getting a cookie.

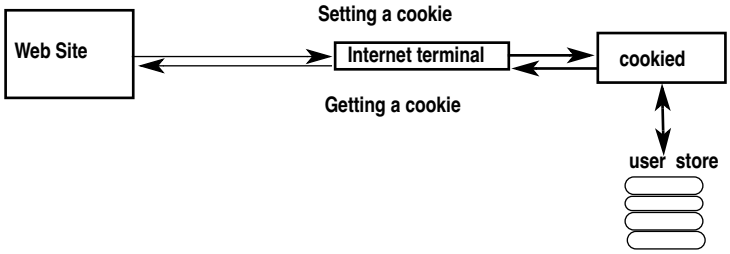


Figure 6-2 Setting and getting a cookie

cookied—cookied is a program that both sets cookies in the user store, and gets cookies for selected web sites.

user store—The user store resides on a NFS (Network File System) storage server, and contains user specific information for several servers. The user store is used by cookied to store cookies.

Cookie Operation

The cookie service has two functions: setting a cookie, and getting a cookie for selected web sites.

Setting a cookie

Some Web sites require registration information before granting a user access. The first time a user goes to one of these web sites and provides the requested information (state information), a Set-Cookie response header is created and sent, using the Internet terminal, to cookied and stored in the user store. The cookie consists of the URL prefix, user name, and an assigned value. A cookie tracking a user's movements within a site is set in the same way. cookied keeps track of state information that arrives via Set-Cookie response headers from each Web site.

Both setting and getting cookie functions for shopping are explained together in the next section.

Getting a cookie

Once a cookie has been set, (in the case of registration, for example) whenever a user goes back to that site, cookied sends the applicable Cookie request header to the Web site.

The cookie can also be used to track a user's movements within a site, so that when the user revisits the site, the user can automatically return to the last location visited. cookied sends the applicable Cookie request header to the Web site.

Some sites (shopping, for example) show the user various products, which they can put in a "shopping cart." A Set-Cookie response header, containing the shopping cart (state information) response header is created and sent, via the Internet terminal, to cookied and stored in the user store.

When the user makes the purchase, the Web site contacts cookied using the Internet terminal. cookied retrieves the shopping cart from the user store and cookied sends the applicable Cookie request header (containing the shopping cart information) to the Web site. In this case, the functions of setting a cookie and getting a cookie are combined.

Special Files

The cookie service produces log files and uses the configuration file to control the operation of the service.

Error Log

cookied.log is used by cookied to record errors, warnings, or other more or less anomylous notifications. It is stored in `~production/webtv/log/cookied.log`.

See "The WebTV Log File Appendix B" for detailed information on error log files.

Events Log

user-events.log is used by all services, including cookied, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/user-events.log`.

Note: The event log is a more complex process than error logging in that it is necessary for different programs to communicate with each other to log events.

See "The WebTV Log File Appendix B" for detailed information on events log files.

Configuration Files

The following sample shows the section of arcadia.cf that controls cookied. There are no options that need to be set or changed by an operations person:

```
<cookied>

     ServeProcessCount 25
          MaxDescriptors 125
     MaxLogFileSize 10240

     ContentRoot /usr/home/production/webtv/content/cookied
     UserRoot /usr/home/production/webtv/userstore/shared/cookied

     RequireNFSUserRoot true

     LogFile /usr/home/production/webtv/log/cookied.log
</cookied>
```

Service Tests

An operations person can execute testcookie for the cookie service to insure that it is running properly on a machine.

testcookie runs the following processes:

- Gets the list of domains that have cookies.
- Gets multiple cookies.
- Adds one cookie.
- Adds multiple cookies.

The following sample shows the section of arcadia.cf that controls testcookie:

```
<testCookie>
     server localhost:1619
</testCookie>
```

Chapter 7

Content Services

Content services include all of the services that a user has access to when using the Internet terminal. This chapter documents each service by providing a description of the components, operation, and special files that it uses. It contains the following sections:

- **Guide Service**
- **Explore Service**
- **Home Service**
- **Content Service**
- **Aroundtown Service**

Guide Service

The Guide service serves sequenced pages of instructions to users. Home, Mail, Explore, AroundTown, and Search all have instruction links in their sidebars that connect to the Guide service.

This section describes the following components:

- **Guide System Components**—describes the components that make up the Guide system.
- **Guide Operation**—describes the operation of the guided.
- **Special Files**—describes the log files produced by guided, and the configuration file and templates used by guided.

Guide System Components

Figure 7-1 illustrates the Guide system. The parts of the system are described following the figure.

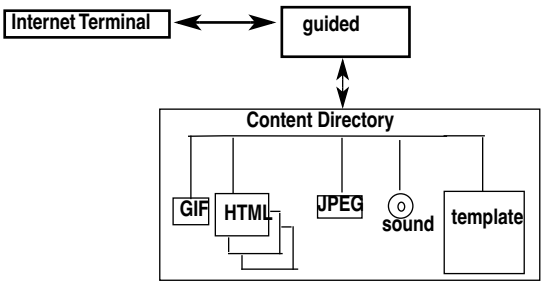


Figure 7-1 Guide Service

guided—Guided is a program that serves content from the content directory to the Internet terminal upon user request.

Content Directory and Templates—The content directory contains standard html files, which can include gifs, jpegs and various sound formats. The content directory also contains template files, which have .tmpl suffixes. Templates contain the formatted html content, and variables that determine the appropriate content to display. Guided serves ordinary html files and executes template pre-processing directives.

Guide Operation

When an Internet terminal requests to be served instructions, guided serves a standard html file from its content tree to the Internet terminal. The Guide service does not interact with any other services.

Special Files

Guided produces log files and uses the configuration file to control the operation of the service.

Error Log

guided.log is used by guided to record errors, warnings, or other more or less anomalous notifications. It is stored in `~production/webtv/log/guided.log`.

See Appendix B in the *System Operations Manual* for detailed information on error log files.

Events Log

user-events.log is used by all services, including guided, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/user-events.log`.

Note: The event log is a more complex process than error logging in that it is necessary for different programs to communicate with each other to log events.

See Appendix B in the *System Operations Manual* for detailed information on events log files.

Configuration files

The following sample shows the section of arcadia.cf that controls guided. There are no options that need to be set or changed by an operations person:

```
<guided>

     ServeProcessCount 20
          MaxDescriptors 100

     ContentRoot /usr/home/production/webtv/content/guided
    TemplatesRoot /usr/home/production/webtv/content/guided/templates
     LogFile /usr/home/production/webtv/log/guided.log
     ContentRoot /usr/home/production/webtv/content/guided
          SearchURL http://www.excite.com/webtv/query.html
          SearchURL_Blacklist http://www.excite.com/webtv/
query.safe.html
          SearchURL_Yahooligans http://www.yahooligans.com
          ExploreURL_Search_Engines wtv-explore:/
explore?goto_Reference:Search_Tools
</guided>
```

Templates

Guided uses the following templates to serve content to the Internet terminal:

- *AccessControl.tmpl*
- *Alerts.tmpl*
- *CustomerCare.tmpl*
- *Explore.tmpl*
- *Favorites.tmpl**
- *Guide.tmpl*
- *HomePage.tmpl*
- *Instructions.tmpl**
- *Internet.tmpl*
- *Mail.tmpl**
- *NamesAndPasswords.tmpl**
- *News.tmpl*
- *OnScreenKeyboard.tmpl*
- *OneColumn.tmpl*
- *Options.tmpl**
- *Philips.tmpl*
- *PhilipsPrototype.tmpl*
- *PhilipsRemoteControl.tmpl**
- *Searching.tmpl**
- *Sony.tmpl*
- *SonyCodeSet.tmpl**
- *SonyPrototype.tmpl*
- *SonyRemoteControl.tmpl**
- *Transactions.tmpl**
- *TwoColumn.tmpl*
- *WebTV.tmpl*
- *WirelessKeyboard.tmpl*

Explore Service

Explore is a service that serves a three-tiered categorized list of the top 1000 sites that work well on an Internet terminal. This service has three purposes: to determine what content to serve (kidsafe, normal), to serve the categorized list, and to save history data of the last item that a user has visited.

This section describes the following components:

- **Explore System Components**—describes the components that make up the Explore service.

- **Explore Operation**—describes the operation of the Explore service.
- **Special Files**—describes the log files produced by explored, and the configuration file and templates it uses.

Explore System Components

The Explore service is illustrated in Figure 7-2. The parts of the service are described following the figure.

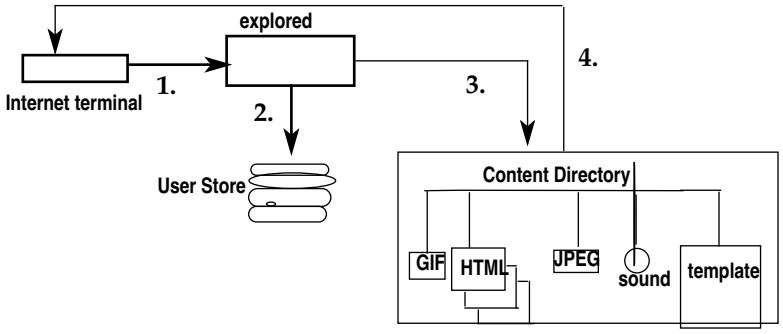


Figure 7-2 Explore service

explored —A program that serves a categorized list of URLs and records the Web pages that a subscriber visits.

User store—The user store is a program that resides on a NFS (Network File System) storage server, and contains user specific information for several servers. explored accesses the user store to see if there is an entry for the user. If there is, the last URL the user visited is displayed.

Content Directory and Templates—The content directory contains standard html files, which can include gifs, jpegs and various sound formats. The content directory also contains template files, which have .tmpl suffixes. Templates contain the formatted html content, and variables that determine the appropriate content to display. explored serves ordinary html files and executes template pre-processing directives.

Explore Operation

When an Internet terminal requests to be served the Explore service, explored determines if the subscriber is operating under kidsafe or normal mode, looks in User Store to see if their is an entry for this subscriber and then displays the categorized list for them. If there is an entry in user store, the last URL that the subscriber visited is highlighted.

Once the subscriber starts visiting Web sites, explored keeps track of each categories and subcategories that they visit and makes an entry in the User Store. explored creates an entry for each category the user visits so that the next time the subscriber visits Explore, the correct category is displayed. If the category name changes or is deleted, the next time the user visits explore, explored defaults to the News/newspapers category.

Special Files

Log Files

Explore uses the following log files to record information about its activities. See "The WebTV Log File Appendix B" in the *Service Operations Reference* for detailed information on log files.

- *explored.log*. This file is used by Explore to record the events associated with the service. It is stored in ~/`production/webtv/log/`.
- *user-events.log* is used by all services, including guided, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/user-events.log`.

Configuration File

The portion of arcadia.cf that controls Explore sets the location in which to find its files and where the log file is placed. No changes will need to be made to the configuration file. A typical production entry is shown below.

```
<explored>
  ServeProcessCount 20
  MaxDescriptors 100

  ContentRoot /usr/home/production/webtv/content/explored
  TemplatesRoot /usr/home/production/webtv/content/explored/templates
  UserRoot /usr/home/production/webtv/userstore/shared/explored

  RequireNFSUserRoot true

  LogFile /usr/home/production/webtv/log/explored.log
</explored>
```

Home Service

The Home service (wtv-home) has one function: it serves the main "Home" page for WebTV subscribers. The Home page is split into several content areas: a static area that contains links to other services and a dynamic area that contains content that changes daily. The dynamic content portion of home currently consists of a feature, aroundtown, and three cool sites, but this will change with each release. The static portion includes links to the setup, mail, favorites, search, and explore services. Figure 7-3 illustrates the parts of a home page.

Figure 7-3 Home Page

This section describes the following components:

- **Home System Components**—describes the components that make up the Home service.
- **Home Operation**—describes the operation of the Home service.
- **Special Files**—describes the log files produced by Home, and the configuration file and templates it uses.

Home System Components

The Home service is illustrated in Figure 7-4. The parts of the system are described following the figure.

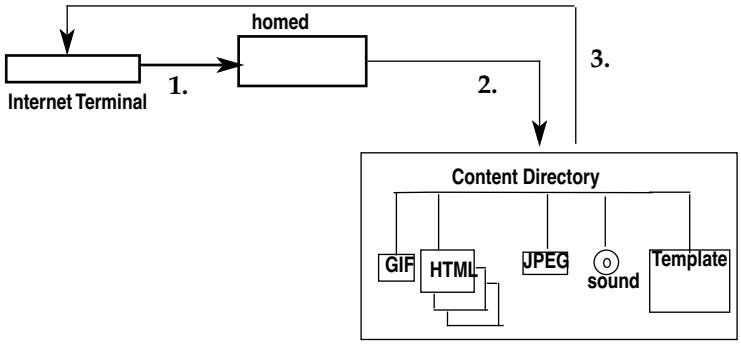


Figure 7-4 Home service

homed—A program that dynamically changes the content to serve based on a subscribers terminal type and user account.

Content Directory and Templates—The content directory contains standard html files, which can include gifs, jpegs and various sound formats. The content directory also contains template files, which have .tmpl suffixes. Templates contain the formatted html content, and variables that determine the appropriate content to display.

Home Operation

When a terminal requests to be served the home page, homed checks to see what type of content a user should be served. This is based on client variables in the home page templates that specify the day of the week, the terminal type, and the account type (kidsafe or normal). The template contains areas that specify what should appear for the dynamic and static portions of the page based on these variables. For example, a 1.0 terminal with a single user will be served "Setup" and "Instructions" in the sidebar. A 1.1 terminal with multiple users has "Setup", "Instructions", and "Switch User" in the sidebar.

Special Files

The home service uses log files and configuration files to log and control the operation of the service.

Log Files

Home uses the following log files to record information about its activities. See "The WebTV Log File Appendix B" in the *Service Operations Reference* for detailed information on log files.

- *homed.log*. This file is used by home to record the events associated with the service. It is stored in `~production/webtv/log`.
- *user-events.log* is used by all services, including guided, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/user-events.log`.

Configuration File

The configuration file contains entries that determine the search URL to use as well as the usual entries for the WebTV services; that is test name, port number, what machine-class to run on, and so on. A typical Production entry is shown below.

```
<aroundtownd>

   ServeProcessCount 20
   MaxDescriptors 100

   PhoneDBFile /usr/home/production/webtv/config/PhoneDB

   ContentRoot /usr/home/production/webtv/content/aroundtownd
   TemplatesRoot /usr/home/production/webtv/content/aroundtownd\
   /templates
   LogFile /usr/home/production/webtv/log/aroundtownd.log
</aroundtownd>
```

Content Service

The Content service serves plain web content to users and has no program functions apart from serving static content. Typical content might include information about the latest upgrade of the WebTV network, a guided tour of the WebTV network, and so on.

This section describes the following components:

- **Content System Components**—describes the components that make up the Content system.
- **Content Operation**—describes the operation of the Content system.
- **Special Files**—describes the log files produced by contentd and the configuration files used by contentd.

Content System Components

Figure 7-5 illustrates the Content system. The parts of the system are described following the figure.

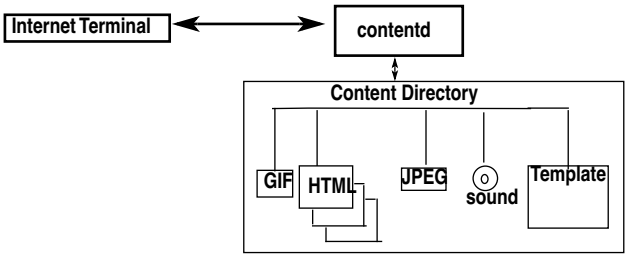


Figure 7-5 Content

contentd—Contentd is a program that serves static content from the content directory to the Internet terminal.

Content Directory and Templates—The content directory contains standard html files, which can include gifs, jpegs and various sound formats. The content directory can also contain template files, which have .tmpl suffixes. Templates contain the formatted html content, and variables that determine the appropriate content to display. Contentd serves ordinary html files and executes template pre-processing directives.

Content Operation

An Internet terminal can request content through the WebTV services that contain hypertext links to the Content server. (Contentd contains only internal content, so an Internet terminal can access content only through the URL links on the wtv-service pages. The wtv-content:path url cannot be typed in by the user.)

When an Internet terminal requests to be served content, contentd serves a standard html file from the content directory to the Internet terminal.

Special Files

Contentd produces log files and uses the configuration file to control the operation of the service.

Error Log

contentd.log is used by contentd to record errors, warnings, or other more or less anomalous notifications. It is stored in `~production/webtv/log/contentd.log`.

See Appendix B in the *System Operation Reference* for detailed information on error log files.

Events Log

user-events.log is used by all services, including Content, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/user-events.log`.

Note: The event log is a more complex process than error logging in that it is necessary for different programs to communicate with each other to log events.

See Appendix B in the *System Operation Reference* for detailed information on events log files.

Configuration File

The following sample shows the section of arcadia.cf that controls contentd. There are no options that need to be set or changed by an operations person:

```
<contentd>

LogFile /usr/home/production/webtv/log/contentd.log
ContentRoot /usr/home/production/webtv/content/contentd
TemplatesRoot /usr/home/production/webtv/content/contentd/templates

ServeProcessCount 20
MaxDescriptors 100

</contentd>
```

Aroundtown Service

The AroundTown service serves content localized to the customer's geographical area. It contains links to outside Web sites and may also include content served from remote Web sites. Aroundtown has a title bar that lists the subscriber's home town.

Figure 7-6 illustrates the Aroundtown page.

Figure 7-6 Aroundtown Page

This section describes the following components:

- **Aroundtown System Components**—describes the components that make up the Aroundtown system.
- **Aroundtown Operation**—describes the operation of the Aroundtown system.
- **Special Files**—describes the log files produced by aroundtownd and the configuration files used by aroundtownd.

- **Service Tests**—describes the service tests that an operations person can run to see if a service is operational.

Aroundtown System Components

The Aroundtown system is illustrated in Figure 7-7. The parts of the system are described following the figure.

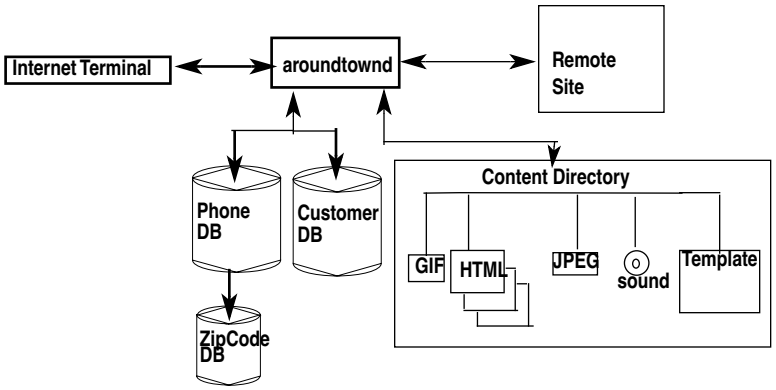


Figure 7-7 AroundTown

aroundtownd—A program that dynamically changes the content to serve based on a subscribers geographical and user information.

Customer DB—The customer database is an Oracle database used by aroundtownd to determine a user's ANI (Automatic Number Identification).

Phone DB—The phone DB is shared by multiple services on a given machine and is used by aroundtownd to determine user's zip codes based on their phone numbers.

Zip Code DB—The zip code DB is a component of aroundtownd and is used to determine the city for a given zip code.

Content Directory and Templates—The content directory contains standard html files, which can include gifs, jpegs and various sound formats. The content directory also contains template files, which have .tmpl suffixes. Templates contain the formatted html content, and variables that determine the appropriate content to display.

Aroundtownd serves ordinary html files and executes template pre-processing directives. The template files include the links aroundtownd needs to get information from remote sites, for example, Acuweather and Yahoo.

Aroundtown Operation

When an Internet terminal requests to be served AroundTown content, aroundtownd looks at the Customer DB to determine the subscriber's Automatic Number Identification (ANI)—the number that is entered into the customer DB when a user first logs on, which is determined by a user's dialin location.

Aroundtownd sends the ANI to the phone DB to determine the zip code, then uses the zip code DB to determine the city name.

Aroundtownd then matches this information to variables in the AroundTown database and determines some of the URLs for the icons featured on its page; for example, MovieLink. If the user requests information from remote sites, for example, Yahoo or Weather, aroundtownd matches this information to variables in the remote sites and downloads the appropriate information to the Internet terminal. So for the Weather area, go to Acuweather and get http Acuweather/zip code.

Special Files

Aroundtownd produces log files and uses the configuration file to control the operation of the service.

Error Log

aroundtownd.log is used by aroundtownd to record errors, warnings, or other more or less anomalous notifications. It is stored in `~production/webtv/log/`*aroundtownd.log*.

See Appendix B in the *Service Operations Reference* for detailed information on error log files.

Events Log

user-events.log is used by all services, including aroundtownd, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/user-events.log`.

Note: The event log is a more complex process than error logging in that it is necessary for different programs to communicate with each other to log events.

See Appendix B in the *Service Operations Reference* for detailed information on the events log file.

Configuration file

The following sample shows the section of arcadia.cf that controls aroundtownd. Remote Page Timeout is the only option that should be set by an operations person.

RemotePageTimeout

Specifies the amount of time to wait before aborting an attempt to download a page from a remote site.

```
<aroundtownd>

ServeProcessCount 20
MaxDescriptors 100

PhoneDBFile /usr/home/production/webtv/config/PhoneDB

RemotePageTimeout        10
ContentRoot /usr/home/production/webtv/content/aroundtownd
TemplatesRoot /usr/home/production/webtv/content/aroundtownd/templates
UserRoot /usr/home/production/webtv/userstore/shared/aroundtownd
LogFile /usr/home/production/webtv/log/aroundtownd.log

CacheRoot /usr/home/production/webtv/spool/aroundtown/cache

; 10,000 cache entries
; 10M target cache size
CacheEntries 10000
TargetCacheSize 10000000

</aroundtownd>
```

Service Tests

An operations person can execute a test for a service to insure that the service is running properly on a machine.

testaroundtown requests a page for a given zip code and validates that the appropriate page is delivered.

The following sample shows the section of arcadia.cf that controls testaroundtown:

```
<testAroundTown>
          server localhost:1628
UserRoot /usr/home/production/webtv/userstore/shared/mail

RequireNFSUserRoot true

</testAroundTown>
```

Chapter 8

Miscellaneous Services

Miscellaneous services include the services that handle changing the configuration of the WebTV services and bookmarking favorite Web pages. This chapter documents each service by providing a description of the components, operation, and special files that it uses. It contains the following sections:

- Favorites Service
- Tricks Service
- Setup Service
- Flashrom Service

Favorites Service

The Favorites service saves and maintains a list of hypertext links to a user's "favorite" web sites (URL's). favorited runs on storefull machines.

Figure 8-1 illustrates the Favorites Page.

Figure 8-1 Favorites Page

This section describes the following components in detail:

- **Favorites System Components**—describes the components that make up the Favorites system.
- **Favorites Operation**—describes the two operations that favorited handles: saving a favorite, and using a favorite.
- **Special Files**—describes the log files produced by favorited, and the configuration files and templates used by favorited.
- **Service Tests**—describes the tests that an operations person can run to see if the Favorites service is operational.

Favorites System Components

The two types of operations that the Favorites system handles are illustrated in Figure 8-2 and Figure 8-3. Figure 8-2 illustrates saving a favorite; and Figure 8-3 illustrates using a favorite to link to a saved web site.

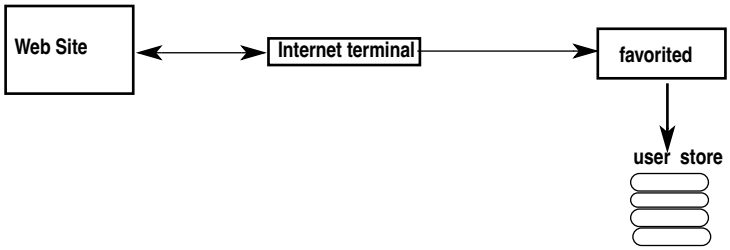


Figure 8-2 Saving a favorite

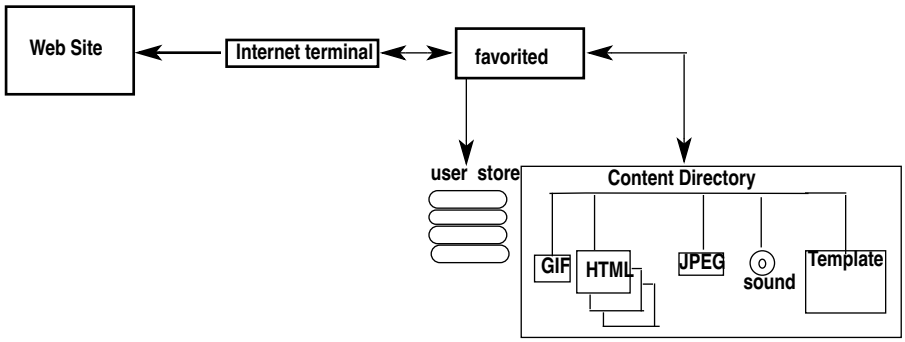


Figure 8-3 Using a favorite to link to a saved web site.

favorited—favorited is a program that both saves favorites, and serves up a list of a favorites.

user store—The user store is a program that resides on a NFS (Network File System) storage server, and contains user specific information for several servers. User store is used by favorited to store and to retrieve favorites.

Content Directory and Templates—The content directory contains standard html files, which can include gifs, jpegs and various sound formats. The content directory can also contain template files, which have .tmpl suffixes. Templates contain the formatted html content, and variables that determine the appropriate content to display. favorited serves ordinary html files and executes template pre-processing directives.

Favorites Operation

favorited has two functions: saving a user's favorite, and serving a list of a user's favorites.

Saving a favorite

When users find a web site they want to save as a favorite, they either press Save on the keyboard, or pull up the options menu and click Save. The URL is sent from the web site, via the Internet terminal, back to favorited, which then stores a hypertext link in the user store.

Using a favorite

When the user clicks on the Favorites icon located on the Home Page, the Internet terminal contacts favorited. favorited retrieves a list of the user's favorites (hypertext links) from the user store and uses a template to send an html page, containing a list of categorized favorites, to the Internet terminal.

Special Files

favorited produces log files and uses configuration files to control the operation of the service.

Error Log

favorited.log is used by favorited to record errors, warnings, or other more or less anomalous notifications. It is stored in `~production/webtv/log/`.

See "The WebTV Log Files in Appendix B in the *Service Operations Reference* for detailed information on error log files.

Events Log

user-events.log is used by all services, including favorited, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/`.

Note: The event log is a more complex process than error logging in that it is necessary for different programs to communicate with each other to log events.

See "The WebTV Log Files" in Appendix B the *Service Operations Reference* for detailed information on events log files.

Configuration File

The following sample shows the section of arcadia.cf that controls favorited. There are no options that need to be set or changed by an operations person:

```
<favorited>

     ServeProcessCount 25
          MaxDescriptors 125

     MaxFavorites 36

     ContentRoot /usr/home/production/webtv/content/favorited
     TemplatesRoot /usr/home/production/webtv/content/favorited/
templates
     UserRoot /usr/home/production/webtv/userstore/shared/favorited

     RequireNFSUserRoot true

     LogFile /usr/home/production/webtv/log/favorited.log
</favorited>
```

Templates

favorited uses the following three templates, which are stored in the favorited content tree:

- *AddFolder.tmpl*—adding a folder
- *Editor.tmpl*—deleting and moving favorites
- *FoldersPage.tmpl*—list of folders
- *Browser.tmpl*—main page
- *Favorite.tmpl*—base template for all pages

Service Tests

An operations person can execute a testFavorite test for the Favorite service to insure that it is running properly on a machine.

testFavorite runs the following processes:

- Goes to a specific URL.
- Goes to the main Favorites page.
- Preloads the Favorites.
- Goes to the Favorites edit page.

The following sample shows the section of arcadia.cf that controls testFavorite:

```
<testFavorite>
     server localhost:1611
</testFavorite>
```

Tricks Service

The Tricks service provides access to URLs that are restricted from non-WebTV users. Normally, internal URLs like wtv-home:/home can't be invoked from non-WebTV content. This is a security feature that prevents hackers from entering the WebTV service. However, the internal URL wtv-tricks is the exception. It provides access to a list of URLs that can be invoked from any html document, or typed into the "Goto" dialog.

The following URLs are currently supported via the Tricks service:

- wtv-tricks:/border—Redirects the WebTV Internet terminal to wtv-setup:/screen-border, which lets the user choose how dark their screen border should be.
- wtv-tricks:/home—Redirects the WebTV Internet terminal to wtv-home:/home, and allows external content to contain a link to the WebTV home page.
- wtv-tricks:/mail—Redirects the WebTV Internet terminal to wtv-mail:/listmail, and allows external content to contain a link to the WebTV mail page.
- wtv-tricks:/search—Redirects the WebTV Internet terminal to the appropriate search page for the user, which varies depending on the KidSafe level.
- wtv-tricks:/goback—Redirects the WebTV Internet terminal to client:goback, and allows external content to contain a link which will cause the WebTV Internet terminal to go back to the previous page.

- wtv-tricks:/tricks—Takes the user to a password-protected page, which gives them a menu of tricky things to do, such as down-loading a specific version of the client software or start a tour.
- wtv-tricks:/info—Displays a page containing interesting, but possibly confusing information about the WebTV Internet terminal and the current connection. This includes things like the Internet terminal's serial number, the phone number of the POP, and the latest phone number the service got through ANI.
- wtv-tricks:/download-demo—Displays a page allowing the user to download a special demo version of the client software used for in-store demos.
- wtv-tricks:/newaddress—Displays an alert asking the user if they want to add a given address to their address book.
- wtv-tricks:/backgroundMusic—Sets the background music from external content.

Special Files

The wtv-tricks service uses log files and configuration files to log and control the operation of the service.

Error Log

tricksd.log is used by tricks to record errors, warnings, or other more or less anomalous notifications. It is stored in `~production/webtv/log/`.

See "The WebTV Log Files in Appendix B in the *Service Operations Reference* for detailed information on error log files.

Events Log

user-events.log is used by all services, including favorited, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/`.

Note: The event log is a more complex process than error logging in that it is necessary for different programs to communicate with each other to log events.

See "The WebTV Log Files" in Appendix B the *Service Operations Reference* for detailed information on events log files.

Configuration File

The only entries in the arcadia.cf file that is specific to the Tricks service are `TricksPasswordExternal` and `TricksPasswordInternal` that are used to set two passwords that can be used to access the wtv-tricks:/tricks page. The idea is that only the external password will be given out, and it will be periodically changed. The following is a typical Production configuration entry.

```
<tricksd>

    ServeProcessCount 4

    ContentRoot /usr/home/production/webtv/content/tricksd
    TemplatesRoot /usr/home/production/webtv/content/tricksd/templates
    LogFile /usr/home/production/webtv/log/tricksd.log
    ContentRoot /usr/home/production/webtv/content/tricksd
    SearchURL http://www.excite.com/webtv/query.html
    SearchURL_Blacklist http://www.excite.com/webtv/query.safe.html
    SearchURL_Yahooligans http://www.yahooligans.com

    TricksPasswordExternal marty
    TricksPasswordInternal xyzzy-plugh-96

</tricksd>
```

Templates

The Tricks service uses two templates:

- *Info.tmpl* which is used to display the wtv-tricks:/info page.
- *Tricks.tmpl* which is used to display the wtv-tricks:/tricks page.

Setup Service

The Setup service allows users to set account and preference information for WebTV Networks. Using the Setup service, the account subscriber (the person billed) can add or change users on an account, as well as set dialing instructions for the Internet terminal.

Users can create their own passwords, if desired, using setupd; and they can control their own WebTV environment. For example, a user can select whether to play background music, adjust text size, use an alphabetical keyboard, set mail options and so on. Users can also enable some advanced options with setupd, such as adjusting the screen to fit their television.

Figure 8-4 illustrates the Setup page.

Figure 8-4 Setup Page

This section describes the following components in detail:

- **Setup System Components**—describes the components that make up the Setup system.
- **Setup Operation**—describes the operation of the Setup system.
- **Special Files**—describes the log files that setupd produces and the configuration files and templates that setupd uses.
- **Service Tests**—describes the tests that an operations person can run on the Setup service to see if it is operational.

Setup System Components

Figure 8-5 illustrates the Setup system. The parts of the system are described following the figure.

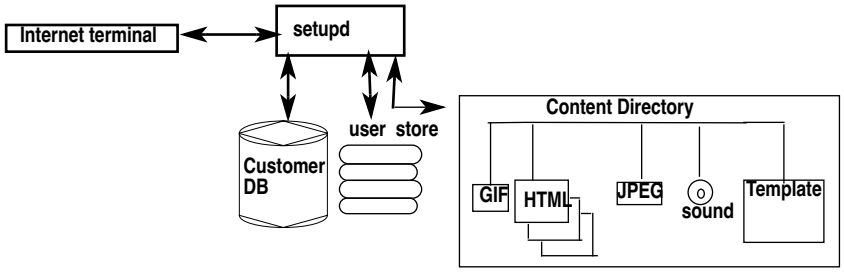


Figure 8-5 Setup

setupd—setupd is a program that stores account information in the customer database and preference information in the user store; and serves account and preference parameters to the Internet terminal when it first logs on.

user store—The user store is a program that resides on a NFS (Network File System) storage server, and contains user specific information for several servers. setupd stores user account preferences in the user store.

Customer Database—The customer database is an Oracle database used by setupd to access customer account information.

Content Directory and Templates—The content directory contains standard html files, which can include gifs, jpegs and various sound formats. The content directory can also contain template files, which have .tmpl suffixes. Templates contain the formatted html content, and variables that determine the appropriate content to display. setupd serves ordinary html files and executes template pre-processing directives.

Setup Operation

When an Internet terminal first logs on, setupd retrieves the account information from the customer database, and the preference parameters from the user store and downloads it to the Internet terminal.

To change account information, the subscriber clicks on the Setup icon on the Home Page. The Internet terminal contacts setupd and setupd sends an html page to the Internet terminal. The user selects the WebTV Users link and is prompted for account information, for example, adding a user. (Only the subscriber is allowed to set account information.) The account information is sent back to setupd and is stored in the user store.

To set preferences, the user clicks on the Setup icon on the Home Page. The Internet terminal contacts setupd and setupd sends an html page to the Internet terminal. The page provides a selection of links to the preferences that can be set. The user selects a link, and setupd downloads another html page prompting the user for preference information. The preference information is sent from the Internet terminal to setupd and is stored in the user store.

Special Files

The setup service produces log files and uses the configuration file to control the operation of the service.

Error Log

setupd.log is used by setupd to record errors, warnings, or other more or less anomalous notifications. It is stored in `~production/webtv/log/`.

See “The WebTV Log Files in Appendix B” for detailed information on error log files.

Events Log

user-events.log is used by all services, including setupd, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/`.

Note: The event log is a more complex process than error logging in that it is necessary for different programs to communicate with each other to log events.

See "The WebTV Log Files in Appendix B" for detailed information on events log files.

Configuration File

The following sample shows the section of arcadia.cf that controls setupd. There are no options that need to be set or changed by an operations person:

```
<setupd>

     ServeProcessCount 25
          MaxDescriptors 125

     ContentRoot /usr/home/production/webtv/content/setupd
    TemplatesRoot /usr/home/production/webtv/content/setupd/templates
     UserRoot /usr/home/production/webtv/userstore/shared/setupd

     RequireNFSUserRoot true

     MailUserRoot /usr/home/production/webtv/userstore/shared/mail

     RequireNFSMailUserRoot true

     LogFile /usr/home/production/webtv/log/setupd.log
     MaxUserCount 6
          MessageWatchTimeSlop 20
          MessageWatchURL wtv-head-waiter:/login?MessageWatch=1
     PhoneDBFile /usr/home/production/webtv/config/PhoneDB
;     ConnectionLog /usr/home/production/webtv/log/setupd.clog
;     ClientSnoop
</setupd>
```

Templates

setupd uses the following templates to display content and prompt users for account and preference information:

- *Accounts.tmpl*
- *AddUserAccess.tmpl*
- *AddUserChat.tmpl*
- *AddUserCommAccess.tmpl*
- *AddUserDone.tmpl*
- *AddUserHumanName.tmpl*
- *AddUserMail.tmpl*
- *AddUserName.tmpl*
- *AddUserPassword.tmpl*
- *AdjustDisplayBrightness.tmpl*
- *AdjustDisplayContrast.tmpl*
- *AdjustDisplayIntro.tmpl*
- *AdjustDisplaySharpness.tmpl*
- *BGSongCategories.tmpl*
- *BGSongCategory.tmpl*
- *ChecksWithTwoBut.tmpl*
- *City.tmpl*
- *DefaultMailbox.tmpl*
- *Display.tmpl*
- *EditSelf.tmpl*
- *EditSelfPassword.tmpl*
- *EditSubCardNumber.tmpl*
- *EditSubCardType.tmpl*
- *EditSubCheck.tmpl*
- *EditSubHumanName.tmpl*
- *EditSubscriber.tmpl*
- *EditSubscriberAddr.tmpl*
- *EditSubscriberPassword.tmpl*
- *EditSubscriberPay.tmpl*
- *EditSubscriberPhone.tmpl*
- *EditUser.tmpl*
- *EditUserAccess.tmpl*
- *EditUserHumanName.tmpl*
- *EditUserMail.tmpl*
- *EditUserName.tmpl*
- *EditUserPassword.tmpl*
- *Keyboard.tmpl*
- *Mail.tmpl*
- *MailAddressing.tmpl*
- *MailMessageWatch.tmpl*
- *MailOrder.tmpl*
- *MailSignature.tmpl*
- *OneColumn.tmpl*
- *Options.tmpl*
- *Phone.tmpl*
- *PhoneAccessNumber.tmpl*
- *PhoneAdvanced.tmpl*
- *PhoneBasic.tmpl*
- *PhoneCallWaitDone.tmpl*
- *PhoneCallWaiting.tmpl*
- *PhoneCallWaitingThreshold.tmpl*
- *PhoneReset.tmpl*
- *Printer.tmpl*
- *RemoveUsers.tmpl*
- *Screen.tmpl*
- *ScreenBorder.tmpl*
- *Setup.tmpl*
- *Sound.tmpl*
- *TCWC.tmpl*
- *Text.tmpl*
- *TwoColWithThreeBut.tmpl*
- *TwoColumn.tmpl*
- *TwoColumnWithContinue.tmpl*
- *TwoColumnWithTwoButton.tmpl*
- *TwoColumnWithoutGoBack.tmpl*
- *TwoRow.tmpl*

Service Tests

An operations person can execute a testSetup test for the Setup service to insure that it is running properly on a machine.

TestSetup gets the list of setup values.

The following sample shows the section of arcadia.cf that controls TestSetup:

```
<testSetup>
     server localhost:1613
</testSetup>
```

Flashrom Service

Code and data for WebTV Internet terminals is stored in two kinds of non-volatile storage, mask ROM (which can't be updated) and flash ROM (which can). The flash ROM download server "flashromd" serves ROM images (which can be old,

developmental or current versions of Internet terminal software) to the Internet terminal for routine upgrades or when the Internet terminal image has been corrupted.

flashromd runs on the storeless, storefull, and mailtod machines that have access to the customer database. This section describes the following components:

- **Flashrom System Components**—describes the components that make up the Flashrom system.
- **Flashrom Operation**—describes the two operations that the Flashrom system handles: upgrades and replacing corrupt images.
- **Special Files**—describes the log files that flashromd produces and the configuration files that flashromd uses.
- **Software Tools**—describes the software tools used by flashromd.
- **Service Tests**—describes the test that an operations person can run on Flashrom to see if it is operational.

Flashrom System Components

Figure 8-6 and Figure 8-7 illustrate the two types of operations that flashromd handles: Figure 8-6 illustrates a typical flash ROM upgrade scenario; Figure 8-7 illustrates the replacement of a corrupted flash ROM.

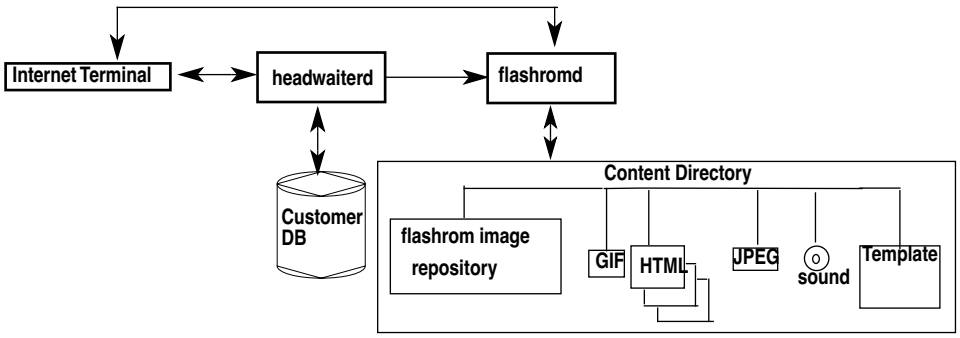


Figure 8-6 Flashrom upgrade

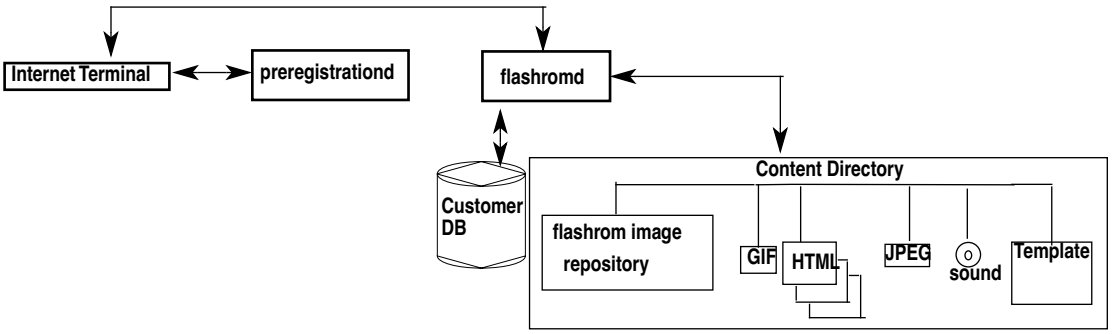


Figure 8-7 Flashrom replacement

flashromd—Flashromd is a program that downloads flash ROM images to the Internet terminal.

headwaiterd—headwaiterd is a program that, in addition to its many other duties, determines whether the Internet terminal gets an upgrade when it logs in.

preregistrationd—preregistrationd is a program that is called when an Internet terminal is unable to contact the WebTV networks.

Customer Database—The customer database is an Oracle database used by flashromd to determine which user group the Internet terminal is in. The user group determines the choice of flash ROM image.

Content Directory and Templates—The content directory contains the flash ROM images, in the flash ROM image repository, and standard html files, which can include gifs, jpegs and various sound formats. The content directory can also contain template files, which have .tmpl suffixes. Templates contain the formatted html content, and variables that determine the appropriate content to display. flashromd downloads flash ROM images, serves ordinary html files and executes template pre-processing directives.

Flashrom Operation

flashromd has two functions: upgrading the flash ROM on the Internet terminal, and replacing a corrupted flash ROM on the Internet terminal.

Upgrading the Flash ROM

To upgrade, when the Internet terminal logs in, headwaiterd determines whether an upgrade is available. If an upgrade is available, headwaiterd sends the Internet terminal to flashromd. If flashromd is ready to provide the upgrade, headwaiterd sends an html page to the Internet terminal either offering the user the option to upgrade, or informing the user that they must upgrade to continue.

If the user agrees to upgrade (“if user agree” case), or if an upgrade is mandatory (“mandatory” case), flashromd sends a special URL to the Internet terminal . This causes the Internet terminal to hang up, reboot from boot ROM, and call back into the flashromd server.

The Internet terminal hands the URL it received back to flashromd (there's no state held in the service, so the box has to be explicit about its request) and flashromd downloads the first "block" of the flash ROM image. flashromd also sends a wtv-visit header, that tells the Internet terminal what block to download next.

When the box has finished downloading a block, and has written it into flash ROM, it sends the wtv-visit URL back to the server for the next block. This continues until the server runs out of blocks. The server does not send down the wtv-visit header with the last block, signaling that the download is complete.

Replacing a Corrupt Flash ROM

If the Flash ROM is corrupted, the Internet terminal calls preregistrationd using the WebTV 800 number. Preregistrationd sends a telly script and the IP address of the flashromd server to the Internet terminal. The Internet terminal hangs up, calls back and connects to the flashromd server. flashromd looks at the customer database to determine what user group the Internet terminal belongs to and begins downloading the appropriate image. From this point, the download process is identical to the Flash ROM upgrade described in the section, "Upgrading the Flash ROM."

Special Files

flashromd produces log files to record information, uses configuration files to control the operation of the service, and uses the flash ROM images and templates to deliver and display content.

Error Log

flashromd.log is used by flashromd to record errors, warnings, or other more or less anomylous notifications. It is stored in `~production/webtv/log/`.

See "The WebTV Log Files in Appendix B" for detailed information on error log files.

Events Log

user-events.log is used by all services, including flashromd, to record every transaction that it does. It is stored in `~production/webtv/log/eventlog/`.

In the sample flashrom log file below, the flash image from the file `.../content/artemis-prototype-000/version-000/bf0app-part000.rom` has been sent to the Internet terminal.

flashromd detected that there were 16 parts (`Pulled parts=16`) to the file, which means that it found the bf0app-rominfo.txt file, and that it is a 1MB image.

```
FlashRom_SendBlock: sending block from
'content/artemis-prototype-000/version-000/bf0app-part000.rom'
FlashRom_SendBlock()       -/0123456789
FlashRom.c:1182             flashromd[4140] 01/07 20:44:54

Pulled parts=16 out of bf0app rominfo file
FlashUtil_ReadRomInfo()    -/0123456789
FlashUtils.c:1263           flashromd[4140] 01/07 20:44:54

next URL is
'wtv-flashrom:/content/artemis-prototype-000/version-000/bf0app-
part001.rom
'
FlashUtil_GetNextURL()     -/0123456789
FlashUtils.c:1197           flashromd[4140] 01/07 20:44:54
```

Note: The event log is a more complex process than error logging in that it is necessary for different programs to communicate with each other to log events.

See "The WebTV Log Files in Appendix B" for detailed information on events log files.

Configuration Files

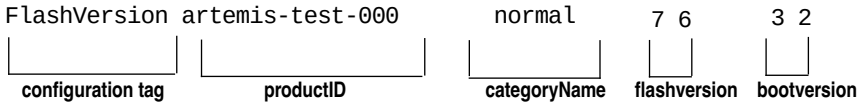
flashromd uses two configuration files: arcadia.cf and flashrom.cf. The following sample shows the section of arcadia.cf that controls flashromd:

```
<flashromd>

; This number can be small, because most of the production servers run
this
; We might lose a little money on each one, but...
     ServeProcessCount 10
          MinIdleProcesses 5

     MaxLogFileSize 10240
     MaxDescriptors 150

     ClientTimeout 120
     LogFile /usr/home/production/webtv/log/flashromd.log
     ContentRoot /usr/home/production/webtv/content/flashromd
     TemplatesRoot /usr/home/production/webtv/content/flashromd/
templates
</flashromd>
```

The “FlashConfigFile” entry in arcadia.cf points the server to “flashrom.cf” to read its configuration file. Items in the flashrom.cf file override any options set in the arcadia.cf file. However, #ifdefs cannot be entered into flashrom.cf.

```
#if defined(PRODUCTION) || defined(TESTDRIVE) || defined(DEMO)
FlashConfigFile ARCADIA_CONFIG(flashrom.cf)
#else
FlashVersion artemis-prototype-000   *       0 0      0 0
FlashVersion sony-webtv-000          *       0 0      0 0
FlashVersion philips-webtv-000        *       0 0      0 0
#endif
```

The following is an example of a FlashVersion configuration entry:



The FlashVersion entry contains the following four parts:

productID—a string that uniquely identifies a WebTV product. The string identifies the manufacturer of the terminal, the type of terminal (test or production), and the version number of the hardware. **Note:** The hardware version number is not used, the value is always 000.

categoryName—is a user-category name, such as “normal”, “gamma”, “pioneer”, or “kiosk”. A categoryName of '*' matches anything.

flashromd runs through the list of FlashVersions until it finds an entry where both the productID and categoryName match. If flashromd can't find a match, it creates an error. If an entry matches, it uses the flashVersion and bootVersion fields to determine if the terminal should do a flashrom download.

flashVersion and **bootVersion**—specify a pair of numbers that indicate the current and minimum flashrom versions that should be installed. The first number in each pair indicates the “current” flashrom version that should be installed, and the second number indicates the “minimum” flashrom version that should be installed.

Internet terminals that don’t have the “current” version are offered an upgrade, and terminals older than the “minimum” version are forced to upgrade. Terminals that have newer than “current” flashrom versions may be forced to upgrade if the current version is flawed, or left alone depending on if the flag “OnlyUpdateOlder” is set to true. If it is set to true the Internet terminal will not be updated.

Production terminals are interested only in the flashVersion numbers. Production terminals use mask ROM for the boot code, and consequently cannot be updated. Production servers use flashrom.cf. A special productID (artemis-test-000) is used for testing.

The following sample shows the flashrom.cf file:

Note: Flashrom.cf header comments pre-date the existence of the bootVersion pair. The "OnlyUpdateOlder" option is left to allow development work to continue without being threatened by updates. NoAutoDownload should be commented out for production unless there is a problem.

```
; For the alpha user group...
FlashVersion artemis-prototype-000 alpha 253 105  0 0
FlashVersion sony-webtv-000 alpha 253 105  0 0
FlashVersion philips-webtv-000 alpha 253 105  0 0

; Now serve everyone else...
FlashVersion artemis-prototype-000 * 110 105  0 0
FlashVersion sony-webtv-000 * 110 105  0 0
FlashVersion philips-webtv-000 * 110 105  0 0

; Set this if you don't want to force boxes with flash versions newer
than
; what's on the server go get flashed.
OnlyUpdateOlder

; If this is set, the headwaiter won't send anybody to the flashrom
daemon
; for automatic downloads.  7-8-6-9, Big Willie's, and braindead flash
; downloads will still work.
;NoAutoDownload>-----
```

FlashROM Images

The flash ROM image is the most important form of content used by flashromd. The following sample shows the flashromd content directory (~/webtv/content/flashromd/), which contains the flash ROM images as well as the template files. The flashromd content directory is described in detail after the example:

```
content/artemis-prototype-000/version-105/{rom-files}
content/artemis-prototype-000/version-110/{rom-files}
content/artemis-prototype-000/version-253/{rom-files}
content/artemis-test-000/version-003/{rom-files}
content/artemis-test-000/version-007/{rom-files}
content/philips-webtv-000/  [ symlink to artemis-prototype-000 ]
content/sony-webtv-000/          [ symlink to artemis-prototype-000 ]
templates/{*.tmpl}
```

The flash ROM image directory (everything under "content") is a hierarchy in which the first path component is the productID and the second is the ROM build version. For example, client v1.0 was build 105, client v1.0.1 was build 110, and client v1.1 (the first public upgrade) was build 253.

The current generation of Sony and Philips Internet terminals use the same ROM images as the WebTV prototype boxes (called "artemis" prototypes for historical reasons), so instead of being separate directories they are just symbolic links to the artemis-prototype-000 directory.

When an Internet terminal requests the "current" version, flashromd takes the productID for that Internet terminal, for example, "sony-webtv-000," for a Sony terminal, finds the user group that the user belongs to, finds a matching

FlashVersion line in the configuration file (the above example gives us current flashVersion=253, minimum flashVersion=105), and concatenates them together to get “sony-webtv-000/version-253/.”

It then takes the Internet terminal type (which is usually “bf0app,” though some older prototypes are "bfeapp"), concatenates that to the previous string and then tacks “-rominfo.txt” onto the end. flashromd then tries to open this file. If the file exists, flashromd gives the go-ahead to the Internet terminal.

Example: content/sony-webtv-000/version-253/bf0app-rominfo.txt

Note: "bfeapp" and "bf0app" are used instead of "artemis-prototype-000" and "artemis-prototype-001" because they are more descriptive and do not imply ordering.

Templates

flashromd uses the following templates, which are stored in the flashromd content tree:

- Fixed content

  Files: FlashOkay.tmpl

  “FlashOkay.tmpl” contains the user interface served when WebTV offers an upgrade or forces a user to upgrade.

  Note: The “force” option forces the Internet terminal to upgrade when not upgrading prevents the Internet terminal from accessing preregistrationd or viewing the Home Page. The force option is not meant to be used after each update.

- Per-version content

  Files: Release251Msg.tmpl
  Release252Msg.tmpl
  Release253Msg.tmpl

  flashromd can send a message to the Internet terminal after an upgrade. The message is tied to a specific flash ROM image build version (e.g. 253), not a client release version (e.g. 1.1).

  To send a message to an Internet terminal after downloading a Flash ROM, add a new template here.

- Internal content

Files: Willie.tmpl
BootDownload.tmpl

Willie.tmpl and BootDownload.tmpl are internal pages for developers and cannot be accessed by users.

Willie.tmpl is the template for Big Willie’s Flash ROM Download Center and is accessible from the “tricks” page. (See network/src/doc/Willie for a full description.)

BootDownload.tmpl is only sent if WebTV Networks is going to send a boot ROM to the Internet terminal.

Note: BootROM cannot be sent to production units.

Software Tools

flashromd does not have any specific software tools; however, the genrom tool is used to split each ROM image (which is anywhere from 1MB to 4MB) into "bite-sized" 128K pieces. The pieces are typically compressed down to about 50% of original size when downloading to the Internet terminal.

Service Tests

An operations person can execute a testFlashRom test for a the FlashROM service to insure that it is running properly on a machine.

testFlashRom runs the following processes:

- Attempts to get the "current" flash image for a "normal" user.
- Attempts to retrieve some of the test images and verifies that is receives the correct part.
- Runs a regression test to verify that upgrades are offered only when they should be (it tries various combinations of "box is older than service," "box is same as service," etc, with different combinations of flags in the flashrom.cf file).

The following sample shows the section of arcadia.cf that controls testFlashRom:

```
<testFlashRom>
     server localhost:1618
</testFlashRom>
```

Glossary

The definitions of the following terms may be helpful in understandingthe WebTV Service document better:

aliasd

An internal service used by sendmail to determine which postoffice should receive an inbound message for a WebTV user.

arcadia.cf

Configuration file that controls the behavior of the WebTV services as well as test each services to make sure that it is opporational.

ANI (Automatic number identification)

A technology for determining the caller's phone number.

contentd

Contentd is a program that serves static content from the content directory to the Internet terminal.

Customer Database

An Oracle database used by WebTV services to access customer account information.

data encryption

WebTV uses a MIME header tag to pass encrypted data from the Internet terminal to the service.

DNS (Domain Name System)

The on-line distributed database system used to map machine names into IP ad-

dresses. DNS servers throughout the Internet use this hierarchical narnespace to allow sites freedom in assigning alphanumeric names and numeric addresses to machines.

Ell TP (Hypertext Transfer Protocol)

HTTP is an application-level protocol idea for distributed, collaborative, hypermedia information systems.

explored

A program that serves a categorized list of URLs and records the Web pages that a subscriber visits.

favorited

A program that both saves favorites, and serves up a list of a favorites.

Firewall

A combination of hardware and software used to separate one or more parts of a network from the rest for security purposes.

FTP

File Transfer Protocol is a tool to move files from another computer site to the Internet terminal.

guided

Guided is a program that serves content from the content directory to the Internet terminal upon user request.

Header

A header appears as the first part of a data packet, and contains addressing, error checking, and encryption information.

headwaiterd

A program that authenticates a user and serves them the list of available services.

HTML template file

Contains HTML and proprietary syntax that the services use to display content on the Internet terminal.

Internet terminal

The hardware box or its internal software being used to access the WebTV network.

IAP (Internet Access Provider)

A company providing WebTV with an Internet information feed.

Invalid URL

When a requested URL is invalid it does not respond to the Internet terminal.

IP (Internet Protocol)

A networking protocol, used to pass information in discrete units called packets, from one machine on the Internet to another. See also TCP/IP and TCP

ISP (Internet Service Provider)

A company providing the hardware and software point-of-presence (POP) permitting the Internet terminal to connect to the WebTV service.

LZPF

A proprietary compression algorithm used to compress HTML.

mail.local.webtv

A program that opens a user's mailbox and delivers mail to it. It also alerts notifyd that mail has arrived.

mailtod

The service that a user's Internet terminal communicates with. mailtod acts on behalf of WebTV users and lists the content of their mailboxes, retrieves their messages, moves messages between mailboxes, and sends messages.

MIME (Multipurpose Internet Mail Exchange)

A standardized method for passing non-text information in email.

notifyd

A program that turns on the red light on the terminal when a postoffice tells the notify service that a user has new mail.

nsproxy

Netscape Proxy software used to handle ftp and gopher requests.

nsproxystarter

Program that starts the nsproxy.

POP (Point-of-Presence)

The local access point to the Internet.

PPP (Point-to-Point Protocol)

A communication protocol that allows a user to connect a modem to the Internet. PPP uses TCP/IP to pass information to and receive information from the Internet.

proxyd

service that retrieves content based on user request and caches, transforms, rewrites HTML, restricts content if necessary, and fixes broken HTML with page patch mechanism.

RAID (Redundant Array of Inexpensive Disks)

A RAID is a magnetic storage device consisting of a number of identical hard drives, all storing the same information. The controller, part of the RAID, ensures that if something happens to the data on one disk, the others maintain the original information.

sendmail

A standard UNIX mail transfer agent (MTA). An MTA is a program that forwards mail between hosts, but it doesn't perform the actual delivery or retrieval of the mail.

Server

A server can either be a machine or a software package running on a particular machine. In either case, the server acts to provide a particular service (for example, email or access to the World Wide Web) to a client. One server machine can run multiple types of server software, providing a client with a large number of services.

Service

A service provided to an Internet terminal by a server. Example services include email, World Wide Web access, and so forth.

Service Down

When a service is either down for maintenance or malfunctioning it does not respond to the Internet terminal.

stard

Stard is a program that serves content to an Internet terminal when a service is either down for maintenance or malfunctioning. Stard also serves content to an Internet terminal requesting an invalid URL.

Subscriber

The person who signs up for WebTV service.

tellyscripts

The programs used by the WebTV terminal to control its functions and customize operating parameters.

templates

Templates, which have .tmpl suffixes, contain the formatted HTML content, and variables that determine the appropriate content to display.

TCP (Transmission Control Protocol)

A networking protocol, used to pass information in discrete units called packets, from one machine on the Internet to another. See also TCP/IP and IP

TCP/IP

A suite, or collection, of communication protocols used by machines on the Internet to communicate with each other and pass information. See also IP and TCP.

URL (Uniform Resource Locator)

A URL is a standardized, universal format for pointing to the location of a file on a local or remote system. URLs typically have the format:

`http://machine.domain.type/directoryl/filename.html"`.

On the World Wide Web, URLs are used as pointers to Web pages.

User

The person currently using the WebTV service.

User Interface

The part of any software thatis presented to the user and allows the user to enter data, make decisions, or otherwise influence the software or hardware's actions.

User store

The user store is a program that resides on a NFS (Network File System) storage server, and contains user specific information for several servers.

WebTV Network

All of the software and hardware that create the WebTV environment.

WTVP (WebTV Protocol)

A modified version of HTTP called WebTV Protocol (WTVP) for use by WebTV devices. HTTP requests are converted to WTVP for optimal speed on the Internet Terminal.

Index

A

access services
headwaiter service 3
logserver service 3
preregistration service 3, 35
registration service 3
star service 3
address book 42
managing 42
admin net 9
administration and monitoring services 2
cored service 6
harvest service 6
service launcher 6
simpled service 6
statusd service 6
aliasd 40, 87
aliasd.log 23
anid.log 23
answering mail 42
arcadia.cf 15
arg0 16
aroundtown service 62
aroundtownd 63
configuration file 64
content directory 63
customer db 63
operation of 63
phone db 63
service tests 65
system components 63
templates 63
user-events.log 64
zip code db 63
aroundtownd 63
aroundtownd.log 23

B

back-end services 2
customer database 7
NetAPP 7
blacklist 3
bootVersion 82

C

caching 47
channel service 5
channeld.log 23
compression algorithms 26

configuration file
arcadia.cf 15
creating 15
file organization 16
global settings 16
possible configurations 16
service settings 16
syntax 17
test settings 17
connection logs 3
connection problems 23
console service 9
content directory 69
content restriction 47
content service 5, 60
components of 60
configuration service 62
content directory 61
contentd 60
contentd.log 61
operation of 61
templates 61
user-events.log 61
content services 2
aroundtown service 5
channel service 5
explore service 5
guide service 4
home service 5
contentd 60
contentd.log 23, 61
cookie 4
cookie service 49
components of 49
configuration file 51
getting a cookie 50
log files 50
setting a cookie 50
cookied.log 22, 50
cored service 6
crashrecord.log 23
customer database 7, 41
customer db 63

D

data encryption 26
db Net 9
DEFINE 20
deleting mail 42
distill-log 43

E

exe 16
explore service 55
log files 57
operation of 56
system components 55
explored 56, 88
explored.log 22

F

favorite service 5
favorited 22, 69
favorited.log 70
favorites service 68
components of 68
configuration file 70
content directory 69
favorited 69
favorited.log 70
operation of 69
saving a 70
templates 69, 71
user store 69
user-events.log 70
using a 70
FILE 20
filter error messages 48
flag 17
flashrom 6
flashrom replacement 79
flashrom service 77
configuration files 81
content directory 79
customer database 79
flashromd 79
flashromd.log 80
headwaiterd 79
images 83
operation of 79
replacing corrupt file 80
system components 78
templates 79, 84
upgrading 79
user-events.log 80
flashrom upgrade 78
flashromd 22, 79
flashromd.log 23, 80
flashVersion 82
forwarding mail 42
ftp requests 3, 46

G

genrom 85
getting a cookie 50
gopher requests 3, 46

guide service 4, 53
configuration files 55
content directory 54
guided 54
guided.log 54
operation of 54
system components 53
templates 54, 55
user-events.log 54, 57, 59
guided 54
guided.log 23

H

harvest service 6
harvestd.log 22, 23
headwaiter service 3, 31
configuration files 34
headwiterd.log 34
operation of 32
sign in process 33
special files 34
system components 31
user-events.log 34
headwaiterd 31, 32, 79
headwaiterd.log 22, 34
home service 5, 57
configuration files 60
content directory 59
homed 59
homed.log 59
operation of 59
templates 59
homed 59
homed.log 23, 59
host 16
html template files 18
HTTP requests 46
http timout 34

I

IAP /ISP net 9
IFDEF 19
IFNDEF 20
image transformations 46
INCLUDE 19

K

kid-friendly accounts 5

L

LABEL 19
listing mail 41
log file 22
client id 23
date and time 24
error message 23
error type 23
example of 24
flag 24
line number 24
parts of 23
process id 24
reporting file 24
reporting function 23
service 24
subscriber id 24
log files
directory structure of 22
format of 23
log problems 23
logging user movements 3
logserver service 3
logserverd.log 3, 22

M

mail services 4
mail store 41
mail system 39
address book 42
aliasd 40
answering mail 42
components of 40
configuration file 42, 43
customer database 41
deleting mail 42
distill-log 43
forwarding mail 42
listing mail 41
log files 42
mail store 41
mail.local.webtv 40
mailtod server 41
mailtod.log 42
moving mail 42
Notify server 41
operation of 41
querydb 43
reading mail 41
receiving mail 41
sending mail 42
sendmail 40
software tools 43
syslog 42
user-events.log 42
mail transfer age 40
mail.local.webtv 40
mail.local.webtv.log 23
mailsorter server 40
mailtod 41
mailtod server 41

mailtod.log 22, 42
major service log files
headwaiterd.log 22
logserverd.log 22
mailtod.log 22
preregistrationd.log 22
proxyd.log 22
max user count 34
messaging services 2
mail service 4
news service 4, 43
notify service 4
miscellaneous services 2
favorite service 5
flashrom service 6
setup service 6
tricks service 6
modem net 8
moving mail 42

N

name 16
NetAPP 7
NetScape proxy 3
network configuration 7
news service 4, 43
nexus net 8
noAdvertise 17
NoHelp 17
nostart 16
notify server 41
notify service 4
notifyd 41, 89
notifying users of problems 3
nsproxy 45, 48
nsproxystarter 4, 48

P

page patching 47, 48
parameterName 18
phone db 63
port 16
postoffice server 40
preregistering users 3
preregistration service 3, 35
preregistrationd.log 22
proxy service 45
caching 47
configuration files 48
content restriction 47
ftp requests 46
gopher requests 46
HTML rewrite 47
HTTP requests 46
log files 48
nsproxy 45
nsproxystarter 45, 48
operation of 45
page patching 47, 48
proxyd 45
transformation of data 46
transformations of content 46
websim 48
proxy services
cookie 4
filters 3
Netscape proxy 3
nsproxystarter 4
proxy 3
proxyd 45
proxyd.log 22, 48

Q

querydb 43

R

reading mail 41
receiving mail 41
registerd.log 22
registering users 3
registration 10
registration service 3
RemotePageTimeout 65

S

searchd.log 23
sending mail 42
sendmail 40, 90
service configuration 10
service files
configuration file 15
log files 15
template files 15
service launcher 6

service log files
aliasd.log 23
anid.log 23
contentd.log 61
cookied.log 22, 50
explored.log 22, 57
favorited.log 22, 70
flashromd.log 22, 23, 80
guided.log 23
harvestd.log 22, 23
headwaiterd.log 22, 34
homed.log 23, 59
logserverd.log 22
mail.local.webtv.log 23
mailtod.log 22, 42
ns-proxy.log 48
preregistrationd.log 22
proxy.log 48
proxyd.log 22, 48
registerd.log 22
servicelauncher.log 22, 23
stard.log 36
statusd.log 22, 23
syslog 42
tricks.log 23
tricksd.log 23, 72
user-events.log 34, 36, 42, 51, 54, 57, 59, 61, 64, 70, 72, 75, 80
service network
admin net 9
console service 9
db net 9
IAP/ISP net 9
modem net 8
nexus net 8
store net 9
service operation 10
registration 10
service opperation
normal use 12
service setting options
arg0 16
exe 16
flag 17
host 16
noAdvertise 17
NoHelp 17
nostart 16
port 16
service settings
name 16
service tests
testFavorite 71
testFlashRom 85
testSetup 77
servicelauncher 22
servicelauncher.log 23
setting a cookie 50
setup service 6, 73
components of 74
configuration file 76
content directory 75
customer database 75
operation of 75
setupd 75
templates 75, 77
user store 75
user-events.log 75
setupd 22, 75
simpled service 6
standard HTTP headers 26
star service 35
operation of 36
stard.log 36
system components 35
templates 37
user-events.log 36
stard.log 36
statusd 22
statusd service 6
statusd.log 23
store net 9
storefull 22
Surfwatch filters 3
surfwatch filters 48
surfwatchfiltersdir 48
syntax for configuration file 17
syslog 42

T

template files 18
directory structure 18
example of 20
syntax 18
uses of 18
template syntax
DEFINE 20
FILE 20
IFDEF 19
IFNDEF 20
INCLUDE 19
LABEL 19
parameterName 18
test settings 17
testAroundTown 65
testFavorite 71
testFlashRom 85
testSetup 77
transformation of content 46
transformations
image 46
unsupported formats 47

tricks service 6, 71, 77
configuration file 73
supported URLs 71
templates 73
tricksd.log 72
user-events.log 72
tricksd.log 23, 72

U

unavailable.tmpl 37
user preferences 4
user store 32, 69, 75
user-events.log 36, 42, 54, 57, 59, 61, 64, 70, 72, 75, 80

W

websim 48
WebTV services
overview of 2
parts of 2
whitelist 3
WTVP attributes 26
WTVP headers 26
WTVP protocol 25
components of 26
debugging 29
WTVP security 29
WTVP server-signed headers 29
wtv-star service 3

Z

zip code db 63